StreamAlloc: A Framework for Analyzing and Transforming CUDA Code to Enable Asynchronous Execution

SOUMIK KUMAR BASU and JYOTHI VEDURADA, IIT Hyderabad, India

In the CUDA programming model, data transfers on the default stream are synchronous, and, similarly, device kernels launched on the default stream cannot overlap with other kernel computations and data transfers. Overlapping execution can be enabled using asynchronous APIs and streams in CUDA. Using them, however, requires careful handling of data dependencies across multiple data-transfer calls, host operations, and kernel computations to ensure program correctness. Moreover, numerous data transfer calls and kernel calls in a program make it even more challenging to manually assign the appropriate stream identifier for each such call. This challenge remains daunting for non-expert programmers because they lack the right tools and expertise.

To address this, we propose sync2async, a novel optimization technique that transforms synchronous data transfers and kernel launches into non-default-stream asynchronous calls by allocating stream identifiers (and adding stream synchronizations at appropriate places) to maximize parallelizability while preserving dependencies. To identify sync2async opportunities and apply transformations, we introduce StreamAlloc, a data-flow-analysis-based framework with four components: (1) inter-procedural compositional read-write analysis to identify variables read and written at call sites, (2) intra-procedural flow-sensitive Can-Run-Asynchronously (CRA) analysis to detect data-transfer and kernel calls that can run asynchronously, (3) Data Flow Stream Assignment (DFSA) algorithm to schedule such asynchronous calls to different non-default streams, and (4) a transformation framework to apply sync2async and automatically optimize the input program. We have implemented StreamAlloc using LLVM/Clang. On P100, A4000, and A100 GPUs, sync2async achieves geomean speedups of 1.49x, 1.63x, and 2.02x over the baseline, respectively.

CCS Concepts: • **Software and its engineering** → **Automated static analysis**; **Frameworks**; Compilers.

Additional Key Words and Phrases: Stream Allocation, Asynchronous Execution, CUDA Streams, Heterogeneous Computing, Static Analysis, Compiler Optimization

1 INTRODUCTION

GPUs play a crucial role in accelerating computation-intensive tasks across diverse fields [Johnson et al. 2019; Mittal and Vaishay 2019; OpenAI 2023; Redmon et al. 2016; Zhou et al. 2018], including Large Language Models (LLMs), recommendation systems, computer vision, scientific simulations, bioinformatics, and autonomous systems, demonstrating their widespread impact. CUDA (Compute Unified Device Architecture) [CUDA 2025] by NVIDIA stands as a pivotal framework for unlocking the potential of GPUs to accelerate computational tasks. A CUDA program typically comprises a host component running on the CPU (host) and a kernel component running on the GPU (device), with inputs transferred to device memory and outputs returned to the host memory (data transfer component). Most CUDA applications use synchronous data transfers and execute GPU kernels scheduled on the default stream. The default stream operates with a serializing behaviour that mandates the completion of one task before another can commence. While this simplifies the program, it limits the full potential of concurrent execution, leading to suboptimal resource utilization and performance bottlenecks.

Authors' address: Soumik Kumar Basu, cs21resch11004@iith.ac.in; Jyothi Vedurada, jyothiv@cse.iith.ac.in, IIT Hyderabad, Hyderabad, Telangana, India.

This work is licensed under a Creative Commons Attribution 4.0 International License.

```
1  void mvtCuda(){
2  ...
3  cudaMemcpy(a_gpu, a, sizeof(DATA_TYPE) *
       N * N cudaMemcpyHostToDevice);
4  cudaMemcpy(x1_gpu, x1, sizeof(DATA_TYPE)
       * N, cudaMemcpyHostToDevice);
5  cudaMemcpy(x2_gpu, x2, sizeof(DATA_TYPE)
       * N, cudaMemcpyHostToDevice);
6  cudaMemcpy(y_1_gpu, y_1, sizeof(DATA_TYPE
       ) * N, cudaMemcpyHostToDevice);
7  cudaMemcpy(y_2_gpu, y_2, sizeof(DATA_TYPE
       ) * N, cudaMemcpyHostToDevice);
8  ...
9
10 mvt_kernel1<<<grid,block>>>(n, a_gpu,
       x1_gpu,y_1_gpu);
11 mvt_kernel2<<<grid,block>>>(n, a_gpu,
       x2_gpu,y_2_gpu);
12 cudaThreadSynchronize();
13 ...
14 cudaMemcpy(x1_outputFromGpu, x1_gpu,
       sizeof(DATA_TYPE) * N, cudaMemcpy
       DeviceToHost);
15 cudaMemcpy(x2_outputFromGpu, x2_gpu,
       sizeof(DATA_TYPE) * N, cudaMemcpy
       DeviceToHost);
16 }
```

(a) Before Optimization

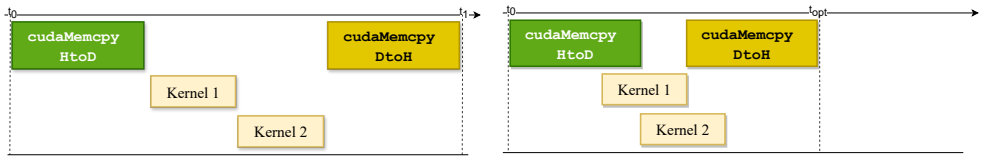
```
1  void mvtCuda(){
2  ...
3  cudaMemcpyAsync(a_gpu, a, sizeof(DATA_TYPE) * N
       * N cudaMemcpyHostToDevice, s1);
4  cudaMemcpyAsync(x1_gpu, x1, sizeof(DATA_TYPE) *
       N, cudaMemcpyHostToDevice, s2);
5  cudaMemcpyAsync(x2_gpu, x2, sizeof(DATA_TYPE) *
       N, cudaMemcpyHostToDevice, s3);
6  cudaMemcpyAsync(y_1_gpu, y_1, sizeof(DATA_TYPE)
       *N, cudaMemcpyHostToDevice, s4);
7  cudaMemcpyAsync(y_2_gpu, y_2, sizeof(DATA_TYPE)
       *N, cudaMemcpyHostToDevice, s5);
8  ...
9  cudaStreamSynchronize(s1, s2, s4);
10 mvt_kernel1<<<grid, block, 0, s6>>>(n, a_gpu,
       x1_gpu,y_1_gpu);
11 cudaStreamSynchronize(s1, s3, s5);
12 mvt_kernel2<<<grid, block, 0, s7>>>(n, a_gpu,
       x2_gpu,y_2_gpu);
13 ...
14 cudaMemcpyAsync(x1_outputFromGpu, x1_gpu, sizeof
       (DATA_TYPE) * N, cudaMemcpyDeviceToHost,
        s6);
15 cudaMemcpyAsync(x2_outputFromGpu, x2_gpu, sizeof
       (DATA_TYPE) * N, cudaMemcpyDeviceToHost,
        s7);
16 }
```

(b) After Optimization

Fig. 1. Sample CUDA code snippet showing blocking calls, host- and device-side computations. While `cudaStreamSynchronize` accepts a single stream identifier, we pass multiple streams for brevity.

Consider an example code snippet in Figure 1a, which is taken from MVT (Matrix-Vector Product and Transpose) program from PolyBench [Grauer-Gray et al. 2012] benchmark suite. The MVT kernel performs a fundamental linear algebra operation involving the multiplication of a matrix by a vector, followed by the transposition of the result. The code uses `cudaMemcpy` calls from Line 3 to Line 7 to perform synchronous host-to-device data transfers. Device kernels at Line 10 and Line 11 perform MVT operations while `cudaMemcpy` calls at Line 14 and Line 15 perform synchronous data transfers from GPU memory to CPU memory. Due to the synchronous nature of each API in the given code snippet, the code exhibits no parallelism between these tasks. However, an opportunity for optimization exists; `mvt_kernel1` can run in parallel with data transfer calls at Line 5 and Line 7 since it is not dependent on the variables transferred in these `cudaMemcpy` calls. Additionally, the execution of `mvt_kernel1` can be performed in parallel with the subsequent kernel, `mvt_kernel2` (Line 11). Despite accessing the same data structure, `a_gpu`, both kernels perform read-only operations on this array, thereby eliminating data dependence concerns. Further, the `cudaMemcpy` operation at Line 14 can run in parallel with kernel `mvt_kernel2`.

To perform multiple data transfers and kernel operations concurrently rather than serially, CUDA provides APIs for asynchronous data transfer and the idea of streams [Streams 2015]. Figures 2a and 2b offer a comparative visualization of the synchronous versus asynchronous behaviour of these operations. As depicted in Figure 2a, the synchronous execution model ensures that data transfers and kernel invocations occur sequentially, resulting in a total execution time denoted by $t_1 - t_0$, which is the cumulative sum of the durations of each individual operation. Conversely, Figure 2b illustrates that by employing asynchronous API calls with appropriate stream allocation, the operations overlap, reducing the total time to $t_{opt} - t_0$, where $t_{opt} < t_1$. This demonstrates the clear efficiency gains of asynchronous execution.



(a) Before Optimization (b) After Optimization

Fig. 2. Execution diagram of memcpy and kernel calls.

To achieve the parallelizability shown in Figure 2b, the programmer can transform API calls (in Figure 1a) at Lines 3, 4, 5, 6, 7, 10, 11, 14, and 15 to asynchronous calls and schedule them into the appropriate CUDA streams, while maintaining dependencies between them. A stream in CUDA is an ordered sequence of operations where functions such as `cudaMemcpyAsync` and kernels associated with a specific stream identifier execute sequentially within the same stream, while operations in different streams can execute concurrently. In particular, two asynchronous DMA (Direct Memory Access) copy engines in a GPU allow simultaneous data transfer across the PCI-Express bus from the CPU to the GPU and back. Additionally, the utilization of Hyper-Q technology facilitates the simultaneous execution of kernels on the GPU device, and the use of pinned memory [NVIDIA 2005] or page-locked memory allocation, in conjunction with stream queues, enables the overlapping of data transfers with computational tasks. Although the programmers are oblivious to such complex underlying hardware features, writing applications using these asynchronous CUDA APIs is still challenging because it requires careful consideration of data dependencies across multiple data-transfer calls, host operations, and device kernel calls to ensure maximum overlap/parallelism and program correctness. It is even more challenging to identify the stream identifiers for each asynchronous call if the application comprises several data transfers and device kernels. For instance, converting synchronous data transfers at Lines 3, 4, 5, 6, and 7 to asynchronous calls, requires stream synchronizations corresponding to Lines 3, 4 and 6 before the GPU kernel, `mvt_kernel1` at Line 10. Similarly, stream synchronizations corresponding to Lines 3, 5 and 7 are also needed before the kernel, `mvt_kernel2` at Line 11.

Given that non-expert programmers often lack the expertise to write optimized programs using streams and asynchronous APIs, an automated framework is essential for effective stream allocation and synchronization placement. Even experienced developers find it tedious to manually identify data and control flow dependencies, leading to potential inefficiencies and errors. For example, GitHub issues from popular real-world open-source projects highlight addressing incorrect manual stream allocation [Rapidsai 2021, 2022], optimizing placement of synchronization barriers [Deepmodeling 2023; Oneflow-Inc 2018, 2022; Rapidsai 2020], and the need for asynchronous API calls at certain program points to enhance performance [Ethereum-Mining 2018; NVIDIA-CCCL 2021; Oneflow-Inc 2021; Rapidsai 2023; Tensorflow 2023]. Such examples highlight the need for automated stream allocation to reduce the complexity and potential errors of manual stream allocation (our evaluation discusses these examples in detail).

To make the life of a programmer easier, several efforts [Beheshti Roui et al. 2020; Cruz et al. 2019; Eyraud-Dubois and Bentes 2020; Jung et al. 2020; Kim et al. 2016; Parravicini et al. 2021; Shekofteh et al. 2020] have been made to (semi-) automatically perform multiple data transfers and kernel operations concurrently from an input code with synchronization calls. Many of these works, along with CUDA Graph API [CudaGraphAPI 2023] by NVIDIA and other scheduling models [Eyraud-Dubois and Bentes 2020; Ghose et al. 2022; Huang et al. 2022; Steinberger et al. 2014], allow programmers to specify multiple tasks as a Directed Acyclic Graph (DAG) to determine the most

effective scheduling; however, this did not resolve the actual problem of manually identifying non-trivial dependencies. Further, several run-time techniques [Cruz et al. 2019; Jung et al. 2020; Kim et al. 2016; Parravicini et al. 2021; Shekofteh et al. 2020] also have been proposed that rely on the operating system's page-fault mechanism or dynamic profile data or hardware changes to identify dependencies at run-time and schedule the tasks into various streams. However, these approaches are complex and can result in run-time overheads.

Motivated by this context, we propose *StreamAlloc*, a data-flow-analysis-based technique to statically analyze an input program and automatically allocate stream identifiers to synchronous data transfers and kernel launches. Thus, it automatically transforms these calls into asynchronous counterparts with appropriate stream synchronizations, maximizing parallelism while preserving program dependencies. We refer to this transformation of code as sync2async optimization. In the example from Figure 1a, StreamAlloc assigns stream identifiers as follows: `s2` and `s4` to Lines 4 and 6, `s6` to the kernel call at Line 10, `s3` and `s5` to Lines 5 and 7, `s7` to Line 11, and `s6` to Line 14 as shown in Figure 1b. This enables parallel execution of `mvt_kernel1` with data transfers at Lines 5 and 7, `mvt_kernel1` with `mvt_kernel2`, and the `cudaMemcpy` at Line 14 with `mvt_kernel2`, achieving the parallelism illustrated in Figure 2b.

To the best of our knowledge, StreamAlloc is the first static analysis technique to automatically identify and apply sync2async opportunities. StreamAlloc is composed of four components: (1) an inter-procedural compositional *read-write* analysis to identify the variables read and written at each call site, (2) an intra-procedural flow-sensitive *Can-Run-Asynchronously (CRA)* analysis to identify which data-transfer and kernel calls can run asynchronously with which others, (3) an automatic scheduler, Data Flow Stream Assignment (DFSA) algorithm that uses the CRA analysis results and schedules the calls to different non-default streams and (4) a transformation pass to automatically apply sync2async optimization to an input program while preserving program dependencies.

We implemented our StreamAlloc using LLVM/Clang [Lattner and Adve 2004; LLVM 2022; Wu et al. 2016] infrastructure. We evaluate our technique on the widely used CUDA benchmark programs such as PolyBench [Grauer-Gray et al. 2012] and HeCBench [Jin and Vetter 2023]. Our results show that our static analysis effectively identifies sync2async opportunities, significantly improving program performance with the streams it suggests. On P100, A4000 and A100 GPUs, our optimization achieves geometric mean speedups of 1.49x, 1.63x, and 2.02x over the unoptimized baseline, respectively.

Contributions

- An intra-procedural flow-sensitive data-flow Can-Run-Asynchronously (CRA) to identify data transfer calls and kernel calls that can run asynchronously on non-default streams
- An inter-procedural compositional read-write analysis to identify variables read and written at each call site, whose results are used to enhance the precision of CRA
- A novel data-flow-analysis-based scheduler that automatically allocates non-default streams for kernel and data transfer calls, using the dependence graph generated by CRA analysis
- A transformation pass that applies sync2async optimization and inserts stream synchronizations based on analysis results for optimal parallelism
- An end-to-end implementation of all the components of StreamAlloc in LLVM/Clang infrastructure to automatically perform sync2async optimization on input CUDA programs
- An extensive evaluation of our approach on three different GPU architectures and comparison with the base code

Program $\mathcal{P}$	=	$s;\ m^{+}$
Method m	=	$\mathcal{H}(\vec{v})\{s_h\} \mid \mathcal{D}(\vec{v})\{s_d\}$
Host-side Statement s_h	=	**call** $\mathcal{H}(\vec{e})\ \mid s_k \mid s_{kst} \mid sync \mid async_mem \mid strm_barrier \mid s$
Device-side Statement s_d	=	$s_k \mid s$
Device Kernel Launch (without Stream) s_k	=	**call** $\mathcal{D}\ \langle\langle\langle\ \vec{e}\rangle\rangle\rangle(\vec{e})$
Device Kernel Launch (with Stream) s_{kst}	=	**call** $\mathcal{D}\ \langle\langle\langle\vec{e}, st\rangle\rangle\rangle(\vec{e})$
Synchronous Call $sync$	=	$sync_mem \mid sync_barrier$
Synchronous Memcpy Call $sync_mem$	=	**call** $m_{sm}(\vec{e})$
Barrier Synchronization Call $sync_barrier$	=	**call** $m_{barrier}()$
Asynchronous Memcpy Call $async_mem$	=	**call** $m_{asm}(\vec{e}, st)$
Stream Synchronization Call $strm_barrier$	=	**call** $m_{stbarrier}(st)$
Statement s	=	$v := e \mid$ **if**$(e)\ \{s\}$ **else**$\{s\} \mid$ **while**(e) **do**$\{s\} \mid s; s \mid$ **skip**
Expression e	=	$v \mid c \mid e_1 \oplus e_2 \mid e_1 \otimes e_2 \mid \neg e \mid e_1 \vee e_2 \mid e_1 \wedge e_2$

$v \in$ **Variable** $\quad c \in$ **Constant** $\quad \vec{v} = (v_1, v_2, \ldots v_n) \quad \vec{e} = (e_1, e_2, \ldots e_n) \quad \oplus \in \{+, -, \times, \div\} \quad \otimes \in \{<, >, =, \geq, \leq, \neq\}$

Fig. 3. Syntax of the CUDA program. Here, st denotes a non-default CUDA stream.

2 FORMALIZATION AND OVERVIEW

2.1 Formalization

Figure 3 presents the syntax of an input CUDA program, which serves as the basis for expressing our static analysis-based framework, StreamAlloc. The program $\mathcal{P}$ consists of statements and methods that can run on a CPU or a GPU. A method $\mathcal{D}(\vec{v})$ runs on the GPU (device [1]) and is commonly called a kernel, while method $\mathcal{H}(\vec{v})$ executes on the CPU (host). (For simplicity, the types are omitted.) Kernel launch requires execution configuration parameters (e.g., thread and threadblock counts) in addition to actual arguments; we abstract configuration parameters using $\langle\langle\langle\vec{e}\rangle\rangle\rangle$ in the device kernel launch statement. A synchronization statement, *sync*, corresponds to either (1) synchronous memory copy operations or (2) explicit barrier synchronization calls. We define *async_mem* to denote the asynchronous memory copy operations and *strm_barrier* to represent a stream synchronization call on a given stream. Formal definitions of these concepts are provided later in this section. Further, we include standard programming constructs, including expressions (e), assignment statements ($v := e$), and control flow constructs, such as branching and loops. Note that we refer to memory copy operations between CPU and GPU as *memcpy* operations or data transfers in the rest of the paper.

While our proposed formalism is general and applicable to various configurations, we restrict our discussion to a single GPU setup for clarity, omitting scenarios involving multiple GPUs attached to a CPU. Furthermore, although GPU kernels predominantly perform computation through memory accesses, our formalization does not explicitly specify them, as memory accesses are automatically converted into register accesses using the `-mem2reg` pass during IR (Intermediate Representation) generation. Section 7 provides further details on the `mem2reg` pass. We now formally define some key terminologies that are necessary for introducing our analysis.

Definition 1 (**Device-side Computation**). *An operation is referred to as* device-side computation *if it is executed within the body of a method $\mathcal{D}$ that runs on a GPU.*

Definition 2 (**Host-side Computation**). *An operation is referred to as host-side computation if it is executed within a method $\mathcal{H}$ that runs on a host system.*

Definition 3 (**Stream**, st). *A CUDA stream st is a sequence of device-side operations (such as kernel launches) and memcpy operations, which execute in the order they are scheduled within st.*

[1] In this paper, we use the words 'GPU' and 'device' interchangeably and, similarly, the words 'host' and 'CPU'.

Definition 4 (**Synchronous Memcpy Call,** *sync_mem*). *In a heterogeneous program $\mathcal{P}$, an API call σ in a host-side method $\mathcal{H}$ is a synchronous memcpy call if it transfers data between host and device memory and blocks the host thread until all device kernels and memcpy operations scheduled across all streams before σ, as well as the current transfer, complete.*

For example, `cudaMemcpy( ...)` API call (Line 3, Figure 1a) is a synchronous memcpy call.

Definition 5 (**Barrier Synchronization Call,** *sync_barrier*). *An API call σ in $\mathcal{H}$ is a barrier synchronization call if it blocks the host thread until all device kernels and memcpy operations scheduled across all streams before σ complete.*

For example, `cudaDeviceSynchronize()` or `cudaThreadSynchronize()` API call (Line 12, Figure 1a) is a barrier synchronization call.

A CUDA program has two types of streams: the default stream and non-default streams. When no explicit stream is specified, kernel launches and data transfers are scheduled on the default stream. Operations scheduled on the default stream cannot overlap with other device computations or data transfers, and further, the default stream ensures all prior operations across all streams are complete before executing the current operation. In contrast, a non-default stream is explicitly created to enable asynchronous execution. Operations within the same non-default stream execute sequentially, while those in different streams may run in parallel, subject to hardware and synchronization constraints. For example, a device kernel launch $\mathcal{D}\ \langle\langle\langle \vec{e} \rangle\rangle\rangle(\vec{e})$ without an explicit stream is scheduled on the default stream, whereas a kernel launch $\mathcal{D}\ \langle\langle\langle \vec{e}, st \rangle\rangle\rangle(\vec{e})$ is scheduled on a non-default stream st. When we refer to streams in this paper, we mean non-default streams unless otherwise stated.

Definition 6 (**Asynchronous Memcpy Call**, *async_mem*). *An API call σ in $\mathcal{H}$ is an asynchronous memcpy call on stream st if it transfers data between host and device memory without blocking the host thread, allowing concurrent execution with device-side kernels and data transfers scheduled on streams other than st and with host-side operations occurring after σ.*

For example, `cudaMemcpyAsync(..., s1)` API call (Line 3, Figure 1b) is an asynchronous memcpy call launched on stream `s1`.

Definition 7 (**Stream Synchronization Call**, *strm_barrier*). *An API call σ in $\mathcal{H}$ is a stream synchronization call on stream st if it blocks the host thread until all device kernels and asynchronous memcpy operations scheduled in st before σ complete.*

For example, `cudaStreamSynchronize(st)` API call (Line 9, Figure 1b) is a stream synchronization call on stream `st`.

Definition 8 (**Data Dependence**, Θ). *Given two program statements s_i and s_j in a method m, we say s_j is data dependent on s_i or vice versa, i.e., $s_i\ \Theta\ s_j$ holds iff*

$$(\mathtt{rd}(s_i) \cap \mathtt{wr}(s_j)) \cup (\mathtt{wr}(s_i) \cap \mathtt{wr}(s_j)) \cup (\mathtt{wr}(s_i) \cap \mathtt{rd}(s_j)) \neq \emptyset$$

where $\mathtt{rd}(s_i)$ and $\mathtt{wr}(s_i)$ denote the set of variables and their aliases (variables which refer to the same memory location) read and written by statement s_i, respectively.

Definition 9 (**sync2async Optimization**). *Transforming a statement s with sync_mem or a device kernel launch (on default stream) in a program $\mathcal{P}$ to a statement s' with async_mem or device kernel launch (on non-default stream) to enable execution of s' in parallel with a set C of host-side computations, device-side computations, and data transfers provided that $\forall c \in C, c\ \Theta\ s'$ does not hold, is called sync2async optimization.*

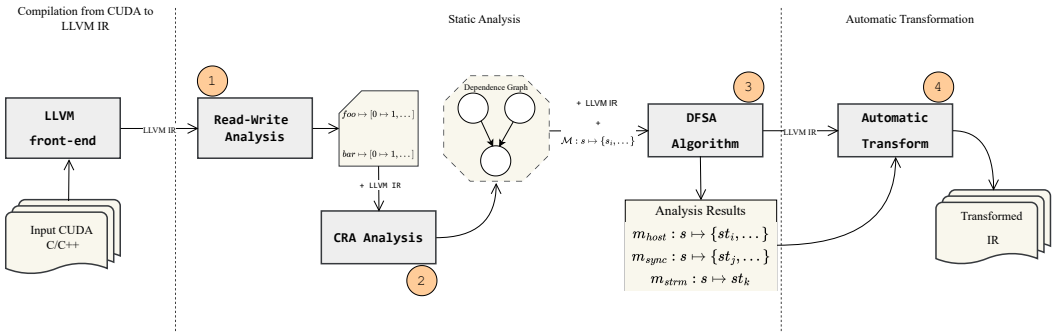


Fig. 4. Block diagram of StreamAlloc.

For instance, a synchronous *sync_mem* call (Line 3, Figure 1a) can be transformed into an *async_mem* call (Line 3, Figure 1b) and a kernel launch without an explicit stream (Line 10, Figure 1a) can be scheduled on a *non-default stream* (Line 10, Figure 1b), allowing parallel execution.

2.2 Overview

Our approach, StreamAlloc (Figure 4), targets two key objectives. Firstly, it aims to transform memcpy calls and device kernel calls from the default stream to non-default streams, enabling parallel execution alongside other host-side computations, device-side computations, and data transfers. Secondly, it aims to effectively allocate stream identifiers to synchronous memcpy calls and device kernel launches, optimizing transformations to maximize parallelism while preserving program correctness.

As shown in Figure 4, our approach comprises a preliminary helper analysis stage followed by two core static analysis stages and a transformation phase. Our Read-Write Analsis pass (Section 3) is an inter-procedural analysis that tracks the variables read and written inside functions compositionally without re-analyzing the callee functions at every call site. Next, our intra-procedural flow-sensitive Can-Run-Asynchronously (CRA) analysis (Section 4) identifies which *sync_mem* and device kernel calls can run asynchronously with which others and it uses the results of Read-Write analysis to avoid overapproximation at call sites. Our Data Flow Stream Assignment (DFSA) algorithm (Section 5) schedules *sync_mem* and device kernel statements identified by CRA analysis to non-default streams and determines the optimal placement and stream identifiers for stream synchronization operations. Finally, the transformation phase (Section 6) automatically modifies the input program by: (1) adding stream allocation and initialization statements, (2) replacing synchronous API calls with their asynchronous counterparts, and (3) inserting stream synchronization statements at appropriate locations. The following sections discuss in detail the three stages and the transformation phase in StreamAlloc.

3 READ-WRITE ANALYSIS

The goal of this analysis pass is to identify which variables are being read and which variables are being written at each call site in a given input program. The results of this analysis will be utilized in the next stage to capture the data dependencies at the call sites precisely. We call it *read-write* analysis pass. It is an inter-procedural compositional bottom-up analysis pass working on the SCC (Strongly Connected Component) call graph. The analysis starts from the leaf node methods in a call graph and analyzes each method independently of its caller method(s). At each method m, the analysis constructs a *summary* map χ that can be composed at the call site statements of its callers.

[Assignment Statement]

$$\frac{s : v := e \quad s \in \texttt{stmt}(m_{curr}) \quad par \in \texttt{param}(m_{curr}) \quad par \odot v \quad \chi[m_{curr} \mapsto \eta]}{\eta[\texttt{index}(par) \mapsto 1]}$$

[Method Call]

$$\frac{s : \textbf{call}\ m(\vec{e}) \quad s \in \texttt{stmt}(m_{curr}) \quad par \in \texttt{param}(m_{curr}) \quad v \in \texttt{variables}(\vec{e}) \quad par \odot v \quad \chi[m \mapsto \eta_m] \quad \eta_m[\texttt{index}(v) \mapsto 1] \quad \chi[m_{curr} \mapsto \eta]}{\eta[\texttt{index}(par) \mapsto 1]}$$

Fig. 5. Transfer function rules for Read-Write analysis.

The key of the summary map is a method m, and the value is another map, with the key being the index of a parameter in m and the value being a `boolean` corresponding to that parameter. If the parameter is read-only in m, its value is 0; if it is written in m, its value is 1. Thus, the `boolean` data-flow fact indicates whether the argument passed by the caller to the respective `param` index of m is being read or written in m.

Figure 5 shows the transfer function rules for read-write analysis. We use a syntax for inference rules similar to those used in past works [Fan et al. 2017; Kuderski et al. 2019; Lei and Sui 2019; Ma et al. 2023; Ye et al. 2014]. The rule [Assignment Statement] outlines the analysis for assignment statements within a method m. The analysis checks whether the assignment statement ($v := e$) writes to any variable (or its alias) that is passed as a parameter par to the current method m_{curr}. The notation $x \odot y$ indicates that the variables x and y are either the same or aliases (may point to the same memory location). If a write is detected, the status of the corresponding index of par in the summary map χ is updated. It is important to note that we do not track reads (R.H.S) in an assignment, as the variables involved are assumed to have default values of '0', implying they have already been read. The confluence operator in our analysis is a bitwise OR (denoted $\vee$), meaning that once a variable is identified as written, its status cannot revert to read.

The rule [Method Call] describes the analysis step encountered at a method call site. The analysis checks whether any parameter of the current method m_{curr} is passed as an argument to a previously analyzed callee method. The status of that parameter variable `par` is then updated by taking the meet of the computed status across all potential callees at that site, accounting for polymorphism. If, at any call site, par is found to be written, the corresponding status of that parameter in the current method is set to '1'. In this way, the computed summaries are composed in a bottom-up manner and the read-write data are computed at every call site in the input program.

4 CAN-RUN-ASYNCHRONOUSLY (CRA) ANALYSIS

To identify if a synchronous data transfer call (*sync_mem*, Definition 4) or a device kernel launched on the default stream can run asynchronously with other host/device/memory-transfer operations, we design an intra-procedural flow-sensitive data-flow analysis called *Can-Run-Asynchronously* (CRA) analysis. We refer to synchronous data transfer calls and device kernels launched on the default stream (no explicit stream specified) as *unstreamed* calls/statements.

Primary Idea. Our data-flow analysis tracks the latest reads and writes to variables by unstreamed calls to identify if a host-side computation or an unstreamed call at a program point depends on any of the unstreamed calls previously occurring in the program flow, following principles similar to reaching definition analysis [Muchnick et al. 1997]. Based on this information, it creates a dependence graph, $G(V, E)$, where vertices V represent unstreamed statements and directed edges E capture RAW (Read After Write), WAR (Write After Read), and WAW (Write After Write) dependencies between them, with the edge direction $u \rightarrow v \in E$ indicating that u must finish before v begins its execution. The dependence graph $G(V, E)$ is the output of our CRA analysis. Our analysis in the next stage (Section 5) uses this graph to allocate streams to unstreamed calls

[Unstreamed-Read]

$$\frac{s : \textbf{call}\ f_{ustrm}(\vec{e}) \quad v_r \in \mathtt{rd}(\vec{e})}{\mu_r^s[v_r \mapsto \{s\}] \quad s \in V}$$

[Unstreamed-Write]

$$\frac{s : \textbf{call}\ f_{ustrm}(\vec{e}) \quad v_w \in \mathtt{wr}(\vec{e})}{\mu_w^s[v_w \mapsto \{s\}] \quad s \in V}$$

[Unstreamed - RAW]

$$\frac{s : \textbf{call}\ f_{ustrm}(\vec{e}) \quad v \in \mathtt{rd}(\vec{e}) \quad \mu_w^s[v \mapsto S_d] \quad s_d \in S_d}{s_d \rightarrow s \in E \quad \mu_r^s[v \mapsto \{s\}]}$$

[Unstreamed - WAR]

$$\frac{s : \textbf{call}\ f_{ustrm}(\vec{e}) \quad v \in \mathtt{wr}(\vec{e}) \quad \mu_r^s[v \mapsto S_d] \quad s_d \in S_d}{s_d \rightarrow s \in E \quad \mu_w^s[v \mapsto \{s\}] \quad v \mapsto S_d \notin \mu_r^s}$$

[Unstreamed - WAW]

$$\frac{s : \textbf{call}\ f_{ustrm}(\vec{e}) \quad v \in \mathtt{wr}(\vec{e}) \quad \mu_w^s[v \mapsto S_d] \quad s_d \in S_d}{s_d \rightarrow s \in E \quad \mu_w^s[v \mapsto \{s\}]}$$

Fig. 6. Transfer function rules for unstreamed statements. Here, the notation, s : **call** $f_{ustrm}(\vec{e})$ denotes unstreamed statements, i.e. synchronous memcpy calls or device kernel launches (without stream). Further, the updates to the dependence graph, $G(V, E)$ are highlighted in blue.

for asynchronous execution and to identify synchronization points with dependent streams to synchronize on for program correctness.

Details of Analysis. The statements of interest for CRA analysis are unstreamed calls and host-side statements. We now discuss in detail the transfer function rules for each of these statements (Figures 6 and 8). Multiple updates under the line in an inference rule indicate updates to various data structures when the constraints above the line hold, illustrating the effect of all transfer functions of a statement. Figure 6 shows the transfer function rules describing the analysis steps for unstreamed calls. For brevity, we represent unstreamed calls (i.e., synchronous memcpy calls and device kernel calls without streams) using function call notation s : **call** $f_{ustrm}(\vec{e})$. We identify the argument variables read or written within function calls, denoted as $\mathtt{rd}(\vec{e})$ and $\mathtt{wr}(\vec{e})$, using read-write analysis (Section 3).

CRA analysis takes the Control Flow Graph (CFG) of a method m as input and generates a dependence graph by iteratively solving data-flow equations at each basic block s to propagate the latest reads (μ_r) and latest writes (μ_w) for the variables accessed by unstreamed calls until a fixed point is reached. The data-flow facts μ_r and μ_w at the exit of s, $Out(s)$, are computed from the entry facts $In(s)$ using the transfer function f_s, given by $Out(s) = f_s(In(s))$, and the rules that define f_s are illustrated in Figures 6 and 8. Entry facts $In(s)$ are computed by joining the μ_r and μ_w from all predecessors of s.

The [Unstreamed-Read] and [Unstreamed-Write] rules track the most recent variable accesses by unstreamed calls by mapping the current unstreamed call s to the variables it reads and writes. Specifically, the rules map s to its read variables in the *latest-read* map (μ_r) and to its written variables in the *latest-write* map (μ_w). The keys of μ_r and μ_w denote accessed variables, while the values record the most recent statements that read or write them. These rules enable dependence detection and the construction of RAW, WAR, and WAW dependence edges when dependencies arise in subsequent statements.

When s reads a variable v already read by a prior unstreamed statement s_p, the analysis updates v's mapping in the latest-read map μ_r to the current statement s (similarly for writes in μ_w). Note that this strong update killing the older mapping of v to s_p occurs only if v and the variable v_p read in s_p are either the same or must-aliases. For may-aliases, instead of killing the older mapping, s is added to the set of latest read statements for both v and v_p. That is, instead of $\mu_r^s[v \mapsto \{s\}]$ [2], the update is $\mu_r^s[v \mapsto S_d \cup \{s\}]$, where S_d is the set of the most recent statements that have read v (and

[2]We represent flow-sensitive information of latest-read map and latest-write map at a program point s using μ_r^s and μ_w^s.

```
1  void main(){
2      ...
3      cudaMemcpy(c_gpu, c, sizeof(DATA_TYPE) * N cudaMemcpyHostToDevice);
4      if(x>y){
5          cudaMemcpy(d_gpu, c, sizeof(DATA_TYPE) * N, cudaMemcpyHostToDevice);
6      }
7      .../* cpu computation */
8      gpu_kernel1<<<grid,block>>>(a_gpu,d_gpu);
9      ...
10     cudaMemcpy(a_output, a_gpu, sizeof(DATA_TYPE) * N, cudaMemcpyDeviceToHost);
11     ...
12     DATA_TYPE x = a_output[0];
13 }
```

Fig. 7. Sample CUDA code snippet to describe CRA analysis.

[Method Call - RAW]

$$\frac{s : \textbf{call}\ \mathcal{H}(\vec{e}) \qquad v \in \mathtt{rd}(\vec{e}) \qquad \mu_w^s[v \mapsto S]}{\mathcal{M}[s \mapsto S] \qquad v \mapsto S \notin \mu_w^s}$$

[Method Call - WAR]

$$\frac{s : \textbf{call}\ \mathcal{H}(\vec{e}) \qquad v \in \mathtt{wr}(\vec{e}) \qquad \mu_r^s[v \mapsto S] \qquad \mathcal{M}[s \mapsto S_o]}{\mathcal{M}[s \mapsto S_o \cup S] \qquad v \mapsto S \notin \mu_r^s}$$

[Method Call - WAW]

$$\frac{s : \textbf{call}\ \mathcal{H}(\vec{e}) \qquad v \in \mathtt{wr}(\vec{e}) \qquad \mu_w^s[v \mapsto S] \qquad \mathcal{M}[s \mapsto S_o]}{\mathcal{M}[s \mapsto S_o \cup S] \qquad v \mapsto S \notin \mu_w^s}$$

Fig. 8. Transfer function rules for host-side function calls. The updation in map $\mathcal{M}$ is highlighted in blue.

similarly for v_p). Similar alias-based updates also apply to other rules. For simplicity and clarity, alias information is omitted from the inference rules, though it is handled in our implementation. Further, our analysis handles branches and loops by performing join operations on data-flow facts μ_r and μ_w from all predecessors at control-flow merge points, using the confluence operator, union.

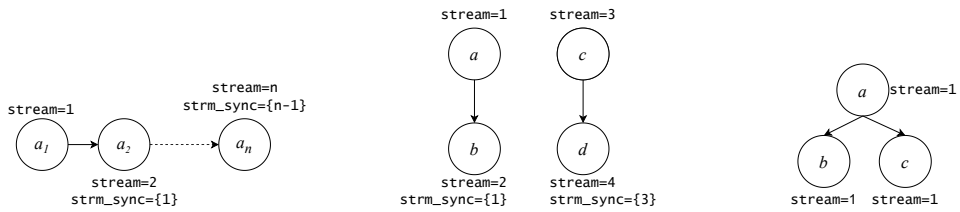
Example. In Figure 7, Line 3 reads variable c and writes to variable c_gpu. Hence, $\mu_r^3[\mathtt{c}]$ and $\mu_w^3[\mathtt{c_gpu}]$ maps to Line 3. Further, Line 5 writes to d_gpu, resulting in $\mu_w^5[\mathtt{d_gpu}]$ mapping to Line 5. Since both lines read variable c, Line 5 replaces Line 3 in $\mu_r^5[\mathtt{c}]$. However, the join operation at Line 7 results in $\mu_r[\mathtt{c}]$ containing both Line 3 and Line 5, i.e., $\mu_r^7[\mathtt{c}] = \{\text{Line 3}, \text{Line 5}\}$. Similarly, μ_w^7 contains $\{\mathtt{c_gpu} \mapsto \{\text{Line 3}\}, \mathtt{d_gpu} \mapsto \{\text{Line 5}\}\}$.

The other three rules in Figure 6 capture data dependencies (RAW, WAR, and WAW) between unstreamed statements. The [Unstreamed - RAW] rule illustrates RAW dependence, where an unstreamed statement s reads a variable v previously written by a predecessor unstreamed statement. Hence, a data dependence edge is added from each writer in $\mu_w^s[v]$ (the latest unstreamed statements that wrote to v) to s, i.e. $\forall s_d \in \mu_w^s[v]\ s_d \longrightarrow s \in E$.

Example. In Figure 7, the kernel computation at Line 8 reads d_gpu, which was written by Line 5. Thus, a dependence edge from Line 5 (in $\mu_w^8[\mathtt{d_gpu}]$) to Line 8 is created: Line 5 $\longrightarrow$ Line 8 $\in E$.

The last two rules [Unstreamed - WAR] and [Unstreamed - WAW] capture WAR and WAW dependencies. When an unstreamed statement s writes to a variable v that was previously read/written by one of its predecessor unstreamed statements, a data dependence edge is created from $\mu_r^s[v]$ (for WAR) or $\mu_w^s[v]$ (for WAW) to s. In the WAR case, v and its latest read statements up to s are removed from μ_r to avoid unnecessary dependence edges with s's successors, as the reads are subsumed by s's write, which holds v's latest value.

We now explain the rules for the host-side program statements (see Figure 8). Due to the similarity in handling various host-side statements, we present only the rules for method calls (rules [Method



(a) Naive approach: Example-1 (b) Naive approach: Example-2 (c) Same stream as parent

Fig. 9. Problems with different stream scheduling techniques.

Call]) for brevity. These rules aim to prevent asynchronous execution of unstreamed calls (after transformation) with host-side computations whenever there are dependencies between them. To achieve this, we construct a map $\mathcal{M}$ that associates a host statement s with the unstreamed calls it depends on. This mapping will allow stream scheduling in the next stage to place appropriate stream synchronizations (Definition 7) before a host statement s, preventing previously scheduled dependent unstreamed calls from running asynchronously with s and ensuring their completion before s. Further, in these rules, we remove the variables (and their mappings) in μ_r and μ_w that the host-side computations in the current statement s depend on because stream synchronization already blocks until the dependent unstreamed calls complete, eliminating the need for subsequent unstreamed statements to include dependence edges from them. We handle predicates in conditionals and loops, expressions in assignment statements and other host-side statements similar to [Method Call] by checking dependencies on their read and write variables.

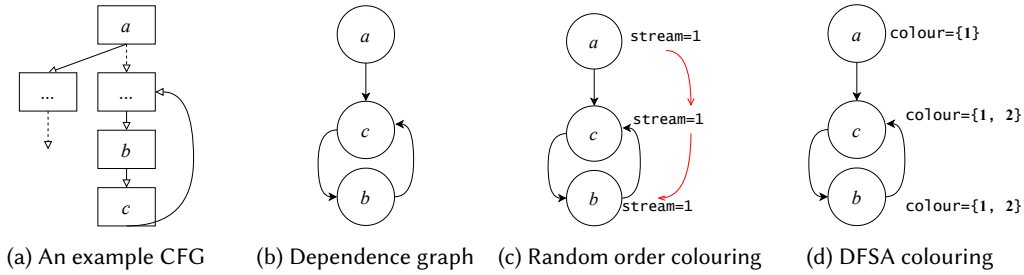
Example. In Figure 7, at Line 12, we have a host-side read access on array `a_output`, and it depends on the write variable of the memcpy call (`a_output`) at Line 10. Hence, our analysis adds an entry Line 12 $\mapsto$ {Line 10} in map $\mathcal{M}$ and removes the entry `a_output` $\mapsto$ Line 10 from μ_w^{12}.

5 STREAM ALLOCATOR

After the CRA analysis generates the dependence graph, we aim to allocate streams to the nodes in the graph, where each node represents an unstreamed call. While there are numerous DAG-based task scheduling algorithms [Bak et al. 2021; Guo et al. 2009, 2010; Maglalang et al. 2017; Marchetti-Spaccamela et al. 2020], we require a new algorithm for stream allocation since, unlike traditional scheduling algorithms that maximize processor utilization while managing job priorities, our objective is to maximize parallel execution while simultaneously minimizing stream synchronization overheads. Additionally, our dependence graph is not a DAG and may contain cycles.

5.1 Problems with Naive Stream Allocation

There are various approaches to address the stream assignment problem. A naive approach involves assigning each node in the dependence graph a unique stream to achieve maximum parallelism. Also, to maintain data dependence, each node should synchronize on its parents' streams. However, this method results in excessive synchronization. For example, consider the scenario depicted in Figure 9a, where a linear dependence exists from node a_1 to node a_n. This naive algorithm may cause all n nodes in the graph to wait on the streams of their predecessors, resulting in excess synchronization compared to assigning a single stream to all nodes and queuing their tasks together. Consider another dependence graph in Figure 9b with nodes a, b, c and d, where node c is a successor of nodes a and b in the CFG and can execute independently from them. However, upon employing the naive algorithm, node b synchronizes with stream 1, forcing a to finish before node c



(a) An example CFG (b) Dependence graph (c) Random order colouring (d) DFSA colouring

Fig. 10. Effect of colouring vertices (allocating stream identifiers) in an order.

can start, thereby restricting parallelism. This issue could be resolved by assigning the same stream to nodes a and b, maintaining their dependence while allowing node c to execute on a different stream independently. Thus, the naive approach triggers excessive stream synchronization and restricts the potential for maximum parallelization.

While assigning a node the same stream as its parent in the dependence graph addresses the above-mentioned issues, this approach still does not achieve optimal parallelism. In the dependence graph shown in Figure 9c, assigning the same stream identifier to nodes b and c as their parent a results in all three nodes sharing the same stream. Nodes b and c are independent and can execute in parallel, but the same stream identifier attaches a false dependence between them, limiting overall parallelism. Thus, it is not optimal for a parent to directly propagate its stream identifier to its child, especially if the child has siblings. Furthermore, assigning streams optimally requires visiting graph nodes in a specific order because random stream propagation from parents can create false dependencies, as demonstrated in Figure 10. Consider a CFG and corresponding dependence graph in Figure 10a and Figure 10b, respectively. In this example, if nodes are visited from a to c to b, then node c receives its stream from its parent a, and similarly, node b inherits its stream from c, which results in nodes b and a sharing the same stream, incorrectly indicating a dependence between b and a that does not exist. Figure 10c illustrates this incorrect stream assignment, with red arrows indicating the propagation direction. If the order were a to b to c (following the statement order in the CFG), then node b would receive a different stream identifier from a because it cannot inherit the identifier from a, as a is not its parent in the dependence graph. Consequently, b and c would share the same stream, while a would have a different one, allowing a and b to execute in parallel. This example highlights the importance of maintaining a specific order for stream allocation to represent dependence relations accurately. Therefore, we develop a novel intra-procedural flow-sensitive data-flow analysis called Data Flow Stream Assignment (DFSA) to allocate (non-default) streams to unstreamed calls to achieve maximum parallelism while enforcing dependencies where needed.

The proposed stream allocation method has two distinct phases: (1) Colour Assignment: each node in the dependence graph is assigned a set of colours, (2) Stream Allocation and Synchronization: the colours assigned in the previous phase are replaced with appropriate stream identifiers, and proper stream synchronizations are identified. Section 5.2 and Section 5.3 detail the working principles of the first phase and second phase of our algorithm, respectively.

5.2 Data Flow Stream Assignment (DFSA) Algorithm

The key idea is to assign each node in the dependence graph a set of colours that will later be replaced with appropriate stream identifiers. To accomplish this, we use an iterative flow-sensitive data-flow algorithm that traverses the CFG of the input program, G', and either generates new colours for nodes or propagates existing colours to each node from their parent nodes in the corresponding

[Assigning New Colour]

$$\frac{\begin{array}{c} s : \textbf{call}\ f_{ustrm}(\vec{e}) \qquad s \in V \cap V' \\ \mathcal{N}_p[s] = \emptyset \ \vee (\mathcal{N}_p[s] \neq \emptyset \wedge \mathcal{F}_p[s] = \emptyset) \\ cnt = \texttt{unique_col()} \qquad s_c \in \mathcal{N}_c[s] \qquad \mathcal{F}_p[s_c \mapsto S_f] \end{array}}{col[s \mapsto \{cnt\}] \quad \texttt{color_child}(s) \quad s \in S_f \quad s \in \Sigma^s_{col}}$$

[Propagation of Colours]

$$\frac{\begin{array}{c} s : \textbf{call}\ f_{ustrm}(\vec{e}) \qquad s \in V \cap V' \\ s_p \in \mathcal{F}_p[s] \qquad col_E[s_p \rightarrow s \mapsto C_e] \\ col[s \mapsto C_s] \qquad s_c \in \mathcal{N}_c[s] \qquad \mathcal{F}_p[s_c \mapsto S_f] \end{array}}{col[s \mapsto C_s \cup C_e] \quad \texttt{color_child}(s) \quad s \in S_f \quad s \in \Sigma^s_{col}}$$

Fig. 11. Transfer function rules for Data Flow Stream Assignment (DFSA). We initialize each entry in the maps $\mathcal{F}_p$ and col with an empty set ($\emptyset$).

dependence graph, G. Specifically, we are interested in the allocation and appropriate propagation of colours across dependence graph nodes, all of which are part of the control-flow graph as well.

In dependence graph G, while the edges themselves convey dependence, the direction of the edges reflects the direction of the control flow. That is, for any two nodes u and v in $G(V, E)$, if there exists an edge from u to v (i.e., $u \rightarrow v \in E$), then there must be a path from u to v in the associated Control Flow Graph $G'(V', E')$, denoted as $u \rightsquigarrow v$. However, conducting data-flow analysis directly on the dependence graph G can lead to incorrect identification of transitive dependencies between nodes. For example, as discussed in Section 5.1, in the dependence graph from Figure 10b, the colour set may be incorrectly propagated from node a to b via c, resulting in all three nodes being assigned the same stream (Figure 10c), but the CFG in Figure 10a shows that node c cannot execute before b. Hence, we perform the iterative flow-sensitive data-flow analysis on the CFG G' by utilizing dependence information from G during the analysis to ensure both adherence to the control flow order and the preservation of data dependencies across unstreamed nodes of G in G'.

Figure 11 shows the transfer function rules describing the steps for allocation and propagation of colours to each node in G. The rule [Assigning New Colour] defines two scenarios under which a new colour is assigned to a node s: (1) if s has no parent nodes, i.e., $\mathcal{N}_p[s] = \emptyset$, or (2) if s has parent nodes, but none have been visited, i.e., $\mathcal{N}_p[s] \neq \emptyset \wedge \mathcal{F}_p[s] = \emptyset$. Here, $\mathcal{N}_p$ is the *parent* map that stores the set of immediate parent nodes [3] for a node s, i.e., $\mathcal{N}_p[s] = \{s_p | s_p \rightarrow s \in E\}$. The *flowed-parent* map, $\mathcal{F}_p$, tracks the parent nodes that have already been visited during the analysis (also means that the parent nodes are coloured), i.e., $\mathcal{F}_p[s] = \{s_p | s_p \rightarrow s \in E \wedge s_p \in \Sigma^s_{col}\}$ where the *coloured-node* set, Σ_{col}, tracks the unstreamed nodes that have been assigned at least one colour during the analysis. In both cases, as no colour can be propagated from parent nodes, a unique integer, *cnt* returned by `unique_col()` procedure, is assigned as the colour set for node s, i.e. $col[s \mapsto \{cnt\}]$. The *node-colour* map *col* stores the colours for each unstreamed node in G. We also insert the current visited node s to the flowed parent set of s_c, where s_c is the child node of s, i.e. $s_c \in \mathcal{N}_c[s] \implies s \in \mathcal{F}_p[s_c]$. Here, $\mathcal{N}_c$ denotes the *child* map that stores the set of immediate child nodes [3] for a node s, i.e., $\mathcal{N}_c[s] = \{s_c | s \rightarrow s_c \in E\}$.

Example. In Figure 10a, node a has no parent nodes in the corresponding dependence graph (Figure 10b), so it is assigned a unique colour, **1** ($col[a \mapsto \{\mathbf{1}\}]$). When node b is first analyzed, only node a has been visited, but a is not an immediate parent of b in the dependence graph (i.e., $\mathcal{F}_p[b] = \emptyset$). As a result, no colour is propagated from a to b, and node b receives a new unique colour, **2** ($col[b \mapsto \{\mathbf{2}\}]$).

DFSA algorithm also propagates the set of colours to its child nodes using the `color_child` procedure, as described in Figure 12. The key idea is that a parent node transfers its colour set to its outgoing edges, and the child nodes inherit the colour sets from their incoming edges. The rule [Single Child] states that if a node s has exactly one child, i.e., $|\mathcal{N}_c[s]| = 1$, the analysis

[3] The terms parent and child nodes refer to the nodes in the dependence graph that also appear in the corresponding CFG.

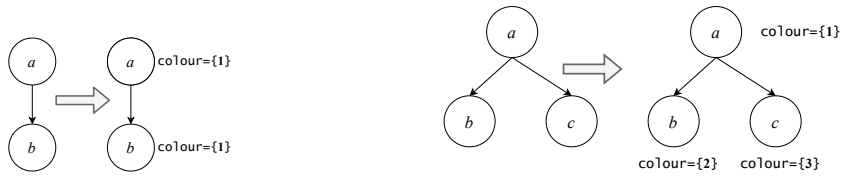
[Single Child]

$$\frac{s_c \in \mathcal{N}_c[s] \qquad |\mathcal{N}_c[s]| = 1 \qquad col[s \mapsto C_v]}{col_E[s \rightarrow s_c \mapsto C_v]}$$

[Multiple Children]

$$\frac{s_c \in \mathcal{N}_c[s] \qquad |\mathcal{N}_c[s]| > 1 \qquad cnt = \texttt{unique_col()}}{col_E[s \rightarrow s_c \mapsto \{cnt\}]}$$

Fig. 12. Inference rules for `color_child` function.



(a) Propagation: single child node

(b) Propagation: multiple children

Fig. 13. Example of different types of colour propagation.

propagates the colour set of node s to the child node s_c via the edge between them. In other words, it assigns the colour set of s to the edge $s \rightarrow s_c$, i.e. $col_E[s \rightarrow s_c \mapsto col[s]]$. The *edge-colour* map col_E stores the colours for edges in G. For example, in Figure 13a, node b inherits the colour set $\{\mathbf{1}\}$ from its parent, node a. The rule [Multiple Children] states that if a node s has multiple children, each child receives a unique colour. This rule prevents multiple child nodes from sharing the same colour, which would hinder potential parallelism between the child nodes. For example, in Figure 13b, node a's children, b and c, are assigned different colours, **2** and **3**, respectively to ensure concurrent execution.

The rule [Propagation of Colours] in Figure 11 shows the scenario where a node inherits colours from its immediate parent nodes that are already assigned colours (propagation of colours from parent nodes to child nodes). That is, if a node s has a non-empty flowed-parent set $\mathcal{F}_p[s]$, its colour set takes the union of the colour sets associated with the incoming edges $s_p \rightarrow s$ for all $s_p \in \mathcal{F}_p[s]$, where these colours are assigned by the above-discussed function `color_child`.

Example. In Figure 10a, when node c is visited, it inherits colour, **1** from node a and colour, **2** from node b, i.e. $col[c \mapsto \{\mathbf{1}, \mathbf{2}\}]$. When node b is revisited (due to a back edge in the CFG), it inherits colours from node c. Thus, the colour set of node c, $\{\mathbf{1}, \mathbf{2}\}$ is propagated to node b, updating b's colour set to include that of c, i.e. $col[b \mapsto \{\mathbf{1}, \mathbf{2}\}]$, as shown in Figure 10d.

In summary, the analysis takes a method's data dependence graph G and CFG G' as input and applies transfer function rules (in Figures 11 and 12) at each basic block to assign colours to nodes in G. It propagates data-flow information using a statement-level coloured-node set Σ_{col}, and method-level maps: the node-colour map col, the edge-colour map col_E, and the flowed-parent map $\mathcal{F}_p$. The analysis terminates when Σ_{col} at each program point does not change, indicating that the colours have been completely propagated to each unstreamed node. After the analysis terminates, we replace the assigned colour sets with stream identifiers to complete the stream allocation and introduce stream synchronization to maintain dependencies as in the dependence graph. We will discuss this process in the following section.

5.3 Allocation of Stream Identifiers

The output of the DFSA is the node-colour map, col, where each node in the dependence graph, G, is assigned one or more colours. Since the unstreamed calls (corresponding to nodes in G) in a given

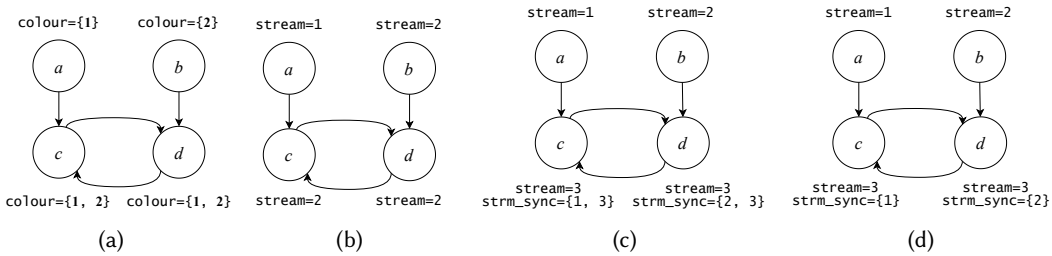


Fig. 14. Stream allocation and insertion of synchronization statements based on assigned colours.

program context can only be executed on a single stream, the multiple colours assigned to nodes must be efficiently mapped to a single stream identifier. A simple approach to achieve this is to select one colour from each colour set as the stream identifier. However, selecting a specific colour, such as the highest or lowest, can lead to suboptimal stream scheduling, where multiple nodes might be assigned the same stream, reducing potential parallelism. For example, in Figure 14a, nodes a, b, c, and d have colour sets $\{\mathbf{1}\}$, $\{\mathbf{2}\}$, $\{\mathbf{1}, \mathbf{2}\}$, and $\{\mathbf{1}, \mathbf{2}\}$, respectively. Selecting the highest colour as a stream identifier from $\{\mathbf{1}, \mathbf{2}\}$ assigns nodes b, c, and d to the same stream (see Figure 14b), while selecting the lowest assigns nodes a, c, and d to the same stream, both reducing parallelism.

To address this limitation, our framework assigns a unique stream identifier to each distinct set of colours. This ensures that even if nodes share similar colours, they are mapped to different streams if necessary, maximizing overlap and computational efficiency.

Example. In Figure 14c, the algorithm maps each distinct colour set to a unique stream identifier: (1) $\{\mathbf{1}\} \mapsto 1$, (2) $\{\mathbf{2}\} \mapsto 2$, and (3) $\{\mathbf{1}, \mathbf{2}\} \mapsto 3$, ensuring optimal stream allocation. Consequently, nodes a and b are allocated to streams 1 and 2, respectively, while nodes c and d are assigned to stream 3, i.e. $m_{strm}[a] = 1$, $m_{strm}[b] = 2$, $m_{strm}[c] = m_{strm}[d] = 3$.

The result of this mapping is a *stream map*, m_{strm}, which records the correspondence between nodes in the dependence graph and their assigned stream identifiers. This map serves as the basis for stream scheduling.

In cases where a node in G depends on multiple parent nodes assigned to different streams, it is necessary to synchronize their execution to ensure that dependent nodes wait for their parent nodes to finish before proceeding. The synchronization is achieved using the CUDA function `cudaStreamSynchronize`, which blocks execution until all preceding tasks in the specified stream are completed (Definition 7). Our algorithm computes a *synchronization map*, m_{sync}, where each node s is associated with the set of parent nodes' streams it must wait on, ensuring correct program execution, i.e., $m_{sync}[s] = \{m_{strm}[s_p] | s_p \rightarrow s \in E\}$. Additionally, when a node's stream is identical to its synchronization stream (the stream it needs to wait for), the synchronization becomes redundant as it gets scheduled in the same queue. In such cases, we eliminate the redundant synchronization stream to maximize efficiency.

Example. In Figure 14c, nodes c and d are allocated to stream 3. Since node c depends on parent nodes a and d, the set of streams it must wait on is $\{1, 3\}$ and similarly, d has to wait on $\{2, 3\}$. However, as c and d are already assigned to stream 3, waiting on the same stream results in redundant synchronizations. Figure 14d shows the updated synchronization sets after removing the redundancy, i.e. $m_{sync}[c] = \{1\}$, and $m_{sync}[d] = \{2\}$ (with $m_{sync}[a] = \{\}$ and $m_{sync}[b] = \{\}$ as they do not depend on any parents).

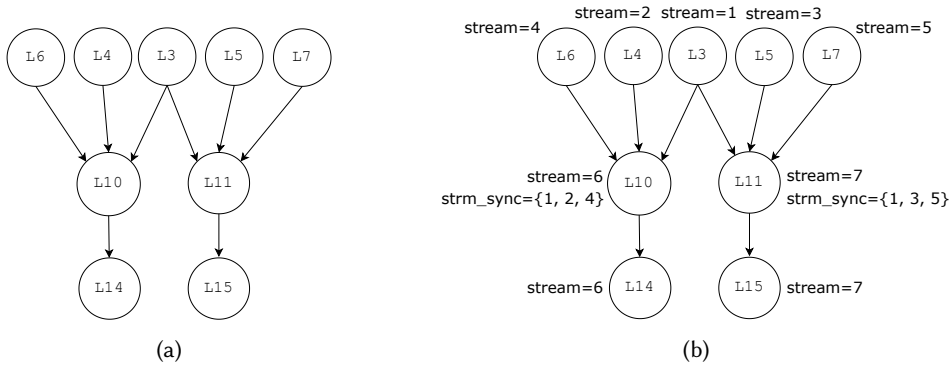


Fig. 15. Dependence graph of MVT (Figure 1a) generated by CRA analysis and effect of DFSA algorithm. Here, node labels represent source code line numbers of the unstreamed calls.

Additionally, host-side statements also need to synchronize with the CUDA streams to handle dependencies. Recall from Section 4 that the map $\mathcal{M}$ contains host-side statements with their corresponding dependent unstreamed call nodes in the dependence graph. A host-side statement must synchronize with all streams of its dependent unstreamed calls in $\mathcal{M}$, ensuring it executes only after the required CUDA streams complete. The output of this step is the *host map* m_{host} that associates each host-side statement with the stream identifiers of its dependent unstreamed calls, i.e. $m_{host}[s_h] = \{m_{strm}[s] | s \in \mathcal{M}[s_h]\}$.

Example. Due to the dependence between the host-side assignment statement (Line 12) and the memcpy call (Line 10) in Figure 7, map $\mathcal{M}$ contains the pair (Line 12, {Line 10}). If the memcpy call at Line 10 is assigned to stream 1, Line 12 must synchronize with stream 1, i.e., $m_{host}[\text{Line 12}] = 1$.

The final output of the stream allocator, shown in Figure 15, demonstrates stream allocation and synchronization for the MVT benchmark program (Figure 1a), with each unstreamed call mapped to its respective stream and synchronization sets. Figure 15a shows the dependence graph with nodes labelled with the source code line numbers of unstreamed calls. The kernel at Line 10 depends on the `HtoDMemcpy` calls at Lines 3, 4, and 6, so its synchronization stream set includes the streams assigned to these calls (see Figure 15b). Similarly synchronization stream set for Line 11 includes the streams allocated to Lines 3, 5, and 7. Further, the kernel calls at Lines 10 and 11 are independent, so they are assigned with different stream identifiers. Lastly, the `DtoHMemcpy` at Lines 14 and 15 are dependent on the kernel calls at Lines 10 and 11 respectively, thereby sharing the same stream identifiers with the respective kernel calls.

Upon completing the stream identifier allocation, the next step is to transform the LLVM IR of the input code, described in the following section.

6 CODE TRANSFORMATION

The procedure for our transformation technique is outlined in Algorithm 1, which describes the sequential steps for performing sync2async optimization (see Definition 9). It takes the stream map m_{strm}, synchronization map m_{sync}, and host map m_{host} as inputs, along with the CFG of the input program. The output is the transformed LLVM Intermediate Representation (modified CFG). The transformation process has three main stages: (1) stream allocation and initialization, (2) insertion of stream synchronization instructions at specified program points, and (3) replacement of synchronous unstreamed calls with their asynchronous equivalents.

Algorithm 1: Algorithm for Code Transformation

1 **Procedure** *Transform*
  Input: CFG G', stream map $m_{strm} : s \mapsto st_k$, synchronization map $m_{sync} : s \mapsto \{st_i, \ldots\}$, host map $m_{host} : s \mapsto \{st_j, \ldots\}$
  Output: Transformed CFG G'_m
2 $G'_m \leftarrow$ Init(G', m_{strm})
3 $G'_m \leftarrow$ Sync$(G'_m, m_{strm}, m_{sync})$
4 $G'_m \leftarrow$ Sync$(G'_m, m_{strm}, m_{host})$
5 **foreach** $s_i \in$ *Keys*(m_{strm}) **do**
6 $s_{async} \leftarrow$ MakeAsync$(s_i, m_{strm}[s_i])$ `// async version of` s_i `on stream` $m_{strm}[s_i]$
7 InsertAfter(s_i, s_{async}) `// insert async call after statement` s_i
8 Remove(s_i) `// remove` s_i `from graph` G'_m
9 **Function** *Init* (G', m_{strm})
10 $G'_m \leftarrow G'$ `// Copy the CFG` G' `into` G'_m
11 $s_{entry} \leftarrow$ EntryNode(G'_m) `// Get the entry basic block of CFG` G'_m
12 **foreach** $s_i \in$ *Keys*(m_{strm}) **do**
13 $s_{alloc} \leftarrow$ Create(`alloca`, $m_{strm}[s_i]$) `// allocate stream variable` $m_{strm}[s_i]$
14 InsertAfter(s_{entry}, s_{alloc})
15 $s_{create} \leftarrow$ CreateCall(`cudaStreamCreate`, $m_{strm}[s_i]$) `// initialize stream variable`
16 InsertAfter(s_{alloc}, s_{create})
17 **return** G'_m
18 **Function** *Sync* (G', m_{strm}, m_{syn})
19 $G'_m \leftarrow G'$
20 **foreach** $s_i \in$ *Keys*(m_{syn}) **do**
21 **foreach** $st_k \in m_{syn}[s_i]$ **do**
22 $s_{sync} \leftarrow$ CreateCall(`cudaStreamSynchronize`, st_k)`// synchronize on stream` st_k
23 $s_{pred} \leftarrow$ ImmediatePred(s_i, G'_m)`// immediate predecessor of` s_i `in graph` G'_m
24 InsertAfter(s_{pred}, s_{sync})
25 **return** G'_m

Initially, our transformation pass generates allocation statements (`alloca` in LLVM IR) for each stream associated with a unstreamed call (see function Init at Line 9, Algorithm 1). A stream variable in the CUDA C/C++ language is of type `cudaStream_t` (correspondingly, `struct.CUstream_st` in LLVM IR), so the `alloca` statements are generated accordingly. Next, `cudaStreamCreate` is used to initialize the streams. The transformation inserts these allocation and corresponding initialization statements at the start of the function (see Lines 14, and 16, Algorithm 1), ensuring stream identifiers are defined and initialized before use.

Example. Figure 16 shows an example of LLVM IR for the MVT program shown in Figure 1a alongside its corresponding transformed IR. Line 2 in Figure 16b shows the generated `alloca` instruction to allocate the stream identifier `s2`. At Line 3, the `cudaStreamCreate` function is inserted to initialize the stream `s2`. The `alloca` statement dominates all subsequent instructions in the function, including the corresponding initialization statement.

Streams can be synchronized using the `cudaStreamSynchronize` function. As detailed in Section 5.3, the maps m_{sync} and m_{host} represent the program points where stream synchronization functions should be placed and the corresponding set of streams to synchronize, respectively. The Sync procedure (see Line 18, Algorithm 1) generates `cudaStreamSynchronize` calls and inserts them at the immediate predecessors of the corresponding program points in the CFG.

```
1  entry:
2  %a_gpu = alloca float*, align 8
3  ...
4
5  ►%call5 = call i32 @cudaMemcpy(
       i8* noundef %6, i8* noundef
        %7, i64 noundef
       1073741824, i32 noundef 1)
6  ...
7
8  ►%call13 = call i32
       @__cudaPushCallConfiguration
       (i64 %27, i32 %29, i64 %33,
        i32 %35, i64 noundef 0, i8
       * noundef null)
9  ...
10
11 ◄%call20 = call i32 @
       cudaThreadSynchronize()
12 ...
13
14 ret void
```

(a) Input LLVM IR

```
1  entry:
2  ►%s2 = alloca %struct.CUstream_st*, align 8
3  ►%call39 = call i32 @cudaStreamCreate(%struct.CUstream_st** %s2)
4  ...
5  %a_gpu = alloca float*, align 8
6  ...
7  ►%load3 = load %struct.CUstream_st*, %struct.CUstream_st** %s2,
       align 8
8  ►%asynccall5 = call i32 @cudaMemcpyAsync(..., i32 1, %struct.
       CUstream_st* %load3)
9  ...
10 ►%load1 = load %struct.CUstream_st*, %struct.CUstream_st** %s1,
       align 8
11 ►%bitcast1 = bitcast %struct.CUstream_st* %load1 to i8*
12 ►%load = load %struct.CUstream_st*, %struct.CUstream_st** %s2,
       align 8
13 ►%call50 = call i32 @cudaStreamSynchronize(%struct.CUstream_st*
       %load)
14 ...
15 ►%call1348 = call i32 @__cudaPushCallConfiguration(..., i64 0,
       i8* %bitcast1)
16 ...
17 ret void
```

(b) Transformed LLVM IR

Fig. 16. Effect of transformation in LLVM IR. In the code, the symbol ► indicates modified instructions, symbol ► indicates newly-generated instructions, and symbol ◄ denotes removed instructions.

Example. In Figure 16a, the GPU kernel call at Line 8 depends on the memcpy statement at Line 5. If the memcpy operation is assigned stream `s2`, the synchronization map associated with Line 8 will include `s2`, i.e., m_{sync}[Line 8] = {`s2`, ...}. Therefore, a `cudaStreamSynchronize` operation on stream `s2` is inserted at Line 13 (in Figure 16b), the immediate predecessor node of the kernel call.

Finally, the transformation pass replaces each node of G'_m with its asynchronous variants (see Lines 6-8, Algorithm 1). Asynchronous CUDA APIs require an additional argument compared to their synchronous counterparts, which specifies the stream identifier (Definition 6). After generating these asynchronous API calls and the necessary initialization statements, the transformation algorithm incorporates the appropriate streams as arguments to these function calls. For certain APIs, such as with `cudaStreamCreate`, streams can be passed directly as arguments. However, other function calls, such as `cudaMemcpyAsync`, may require streams to be loaded into intermediate variables (due to SSA format) or type-cast to different data types (as shown in Line 11 in Figure 16b) before being passed as arguments. This type-casting is required because some CUDA APIs, such as `__cudaPushCallConfiguration`, expect stream arguments to be provided in a generic pointer format (`i8*`), while the stream identifiers are typically allocated as the `struct.CUstream_st` data type. The function `__cudaPushCallConfiguration` is a CUDA runtime API (in LLVM IR) that allows to set the launch configuration (e.g., grid and block dimensions, shared memory size, and stream) for a subsequent kernel launch. Our transformation pass generates these `load` (Line 7, Figure 16b) and `type-cast` (Line 11, Figure 16b) instructions during the post-processing phase. This phase also eliminates unnecessary device synchronization barriers (Line 11, Figure 16a), as our framework handles dependencies through the insertion of appropriate stream synchronizations.

7 EVALUATION

We investigate the following research questions to evaluate our approach StreamAlloc.

RQ1. How efficient are StreamAlloc's analyses and scheduling algorithm in identifying sync2async optimization opportunities?

RQ2. Do open-source CUDA projects employ manual stream allocation for performance optimization, and if so, what issues highlight the need for StreamAlloc?
RQ3. How does sync2async performed using StreamAlloc impact program execution time?
RQ4. How does StreamAlloc compare with CUDA Graph [CudaGraphAPI 2023] scheduling and manual stream allocation?

Implementation. We have implemented our read-write analysis, Can-Run-Asynchronously (CRA) analysis, Data Flow Stream Assignment (DFSA) algorithm, and transformation pass in the LLVM/Clang-14.x [Lattner and Adve 2004; LLVM 2022; Wu et al. 2016] infrastructure. Our CRA analysis works on the SSA (Static Single Assignment) representation obtained using `-mem2reg` flag (which promotes memory references into register references, resulting in a "pruned" SSA form).

Experimental Setup. We use NVIDIA Tesla P100 (Pascal architecture), NVIDIA RTX A4000 (Ampere architecture), and NVIDIA Tesla A100 (Ampere architecture) GPUs for our experiments. The P100, A4000, and A100 machines have 16GB, 16GB, and 80GB device memory, and 20 Intel(R) Xeon(R) Silver, 64 AMD EPYC 7452, and 64 Intel(R) Xeon(R) Platinum CPU physical cores, respectively. In all these machines, CUDA version 11.1 has been used.

Benchmarks. To rigorously evaluate StreamAlloc with respect to diverse real-world scenarios, we use benchmark programs from two different established benchmark suites: PolyBench [Grauer-Gray et al. 2012] and HeCBench [Jin and Vetter 2023]. The PolyBench programs fall into four major categories: (1) Convolution, (2) Linear Algebra, (3) Data mining, and (4) Stencils. HeCBench provides a comprehensive set of heterogeneous computing benchmark programs implemented in CUDA [CUDA 2025], HIP [Babej and Jääskeläinen 2020], SYCL [Khronos 2020], and OpenMP [Dagum and Menon 1998], covering diverse domains such as physics, linear algebra, and machine learning. These programs are primarily sourced from other reputable benchmarks, including Rodinia [Che et al. 2009], NPB [Araujo et al. 2020], N-body simulation [Playne et al. 2009], and Hetero Mark [Sun et al. 2016]. We exclude programs that lack more than one device kernel launch, as they do not present any optimization opportunities, and report only the benchmark programs from PolyBench and HeCBench with sync2async opportunities.

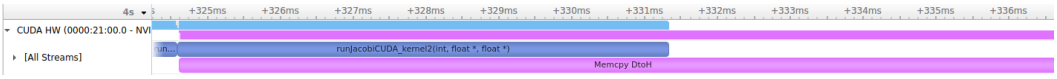
In RQ2, we investigate real-world open-source projects on GitHub to identify manual stream allocation and its associated challenges. Further in RQ4, we evaluate our framework on a set of six benchmarks taken from an open-source benchmark suite GrCUDA [Parravicini et al. 2021], that was created to compare Parravicini et al. [2021]'s work with CUDA Graph. These benchmark programs include complex real-world computations such as Vector Square (`vec`), Black & Scholes (`b&s`), Image Processing (`image`), Machine Learning Ensemble (`ml`), Hyperlink-Induced Topic Search (`hits`), and Deep Learning (`dl`).

7.1 RQ1. Efficacy of StreamAlloc

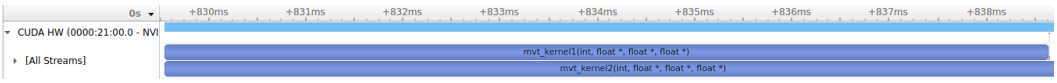
Figure 17 shows the results of our static analysis on PolyBench and HeCBench benchmark suites. StreamAlloc identified 33 optimization opportunities across 26 benchmark programs for performing sync2async optimization. The third column, *#ustrm*, shows the total number of unstreamed calls present in the benchmark programs, while the fourth column, *#strm*, shows the total number of streams assigned by StreamAlloc for each benchmark. In particular, *#strm* is lower than *#ustrm* in several benchmark programs, highlighting that our scheduling algorithm effectively consolidates stream allocations rather than naively assigning a separate stream to each unstreamed call, which would require additional synchronization to handle dependencies (Section 5).

Suite	Benchmark	#ustrm	#strm	O_{type}		T_{anlys}(ms)	T_{trans}(ms)
				KM	KK		
PolyBench	2mm	8	7	✓	✗	49	31
	3mm	11	10	✓	✓	63	33
	adi	11	11	✓	✗	99	40
	atax	7	6	✓	✗	41	27
	bicg	9	7	✓	✓	45	28
	correlation	10	8	✓	✗	65	34
	covariance	7	6	✓	✗	46	30
	doitgen	6	6	✓	✗	45	28
	fdtd2d	8	8	✓	✓	57	31
	gemver	13	12	✓	✗	68	34
	jacobi1d	6	6	✓	✗	38	26
	jacobi2d	6	6	✓	✗	41	28
	mvt	9	7	✓	✓	41	27
HeCBench	ace	11	11	✓	✓	283	36
	asmooth	9	7	✓	✗	117	39
	attention	8	8	✓	✗	68	28
	compute-score	9	9	✓	✗	126	73
	entropy	5	5	✓	✗	71	34
	gaussian	8	8	✓	✗	70	37
	intrinsic	6	4	✓	✗	62	28
	is	15	13	✓	✓	817	50
	knn	7	5	✓	✗	62	36
	mis	5	5	✓	✗	50	33
	pnpoly	6	6	✓	✗	83	41
	reaction	27	27	✓	✓	1461	65
	rsmt	12	12	✓	✗	184	132

Fig. 17. Static analysis results on benchmark suites (Time measured in milliseconds).

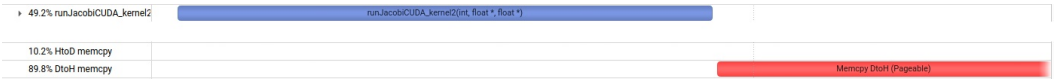


(a) Example of **KM** overlap in `jacobi2d` program



(b) Example of **KK** overlap in `mvt` program

Fig. 18. Timeline diagram of different overlaps in transformed CUDA programs.



(a) Execution of kernel and memcpy in `jacobi2d` program (unstreamed)

(b) Execution of GPU kernels in `mvt` program (unstreamed)

Fig. 19. Timeline diagram of kernel executions and data transfers in unstreamed CUDA programs.

The fifth column, O_{type}, categorizes the identified sync2async opportunities in the benchmark programs. Overlapping computations and data transfers that occur from sync2async opportunities

can be classified into four types: **KM** (Kernel-Memcpy overlap), **KK** (Kernel-Kernel overlap), **MM** (Memcpy-Memcpy overlap), and **CU** (CPU computation-Unstreamed overlap). Specifically, **KM** overlaps occur when a device kernel computation overlaps with a data transfer between the CPU and GPU, while **KK** overlaps reflect concurrent execution of multiple device kernels. **MM** overlaps involve simultaneous opposite-direction data transfers, and **CU** overlaps happen when kernel computation or data transfer overlap with host-side computation. We used the NVIDIA NSight profiler [NVIDIA 2024] to observe **KK** and **KM** overlaps during program execution on the A4000 GPU. Figure 17 only reports **KM** and **KK** overlaps since: (1) **MM** overlaps are not observed in any benchmark program, and (2) **CU** overlaps cannot be visualized by the NVIDIA profiler. A tick mark (✓) in the **KM** or **KK** column indicates the presence of the corresponding overlap, while a cross mark (✗) denotes its absence. As shown in the table, our analysis precisely identifies optimization opportunities in all benchmark programs, validated by manual analysis and by comparing execution timelines of optimized and baseline programs using the NSight Profiler.

Figures 18a and 18b show the screenshots captured from the NSight UI to illustrate **KM** and **KK** overlaps. Figure 18a shows the timeline diagram illustrating the overlap of a complete device kernel computation with Device-to-Host data transfer (**KM** overlap) in the optimized '`jacobi2d`' program. Similarly, Figure 18b shows an overlap between two different device kernel computations (**KK** overlap) in the optimized '`mvt`' benchmark program. In these figures, the blue horizontal bar represents the device kernel execution time, while the pink bar indicates the Device-to-Host data transfer time. The figures show successful overlap in the optimized programs, indicated by overlapping horizontal bars by NSight. In contrast, in the unstreamed versions (see Figure 19), the bars are side by side, indicating no runtime overlap. Further, profiling indicates that the device kernel operation incurs a minor additional overhead compared to its unstreamed version under the **KK** and **KM** overlap. In **KM** overlap, kernels experience a slight increase in execution time due to contention with concurrent data transfers for global memory bandwidth. However, data transfer durations show no additional overhead as they are handled by dedicated DMA engines using large contiguous transfers to and from page-locked (pinned) memory. In **KK** overlap, the kernel launched second shows a slight increase in runtime because of contention for shared compute or memory resources. These additional overheads are minimal (around 3–4% of the kernel execution time) due to fine-grained warp scheduling and ample memory bandwidth in modern GPU architectures, and are consistently outweighed by the aggregate performance gains (see Section 7.4).

All programs in the benchmark suites exhibit **KM** overlap opportunities, while seven programs demonstrate instances of **KK** overlap. The **KK** overlap can arise in three forms: (1) both kernels are compute-bound, (2) both kernels are memory-bound, or (3) one kernel is compute-bound and the other is memory-bound. According to a prior work [Shekofteh et al. 2020], **KK** overlap may not be significant in runtime, if both kernels are of the same type, as they compete for identical hardware resources. The most favourable **KK** overlap case occurs when a compute-bound kernel overlaps with a memory-bound one, as this complementary behaviour exploits distinct hardware resources. However, none of the **KK** overlap instances observed in our benchmarks exhibit such complementary workload behaviour; each **KK** overlap contained the kernels having more memory operations than the computation operations. Notably, although these kernels are dominated by memory operations, they do not saturate the available memory bandwidth. As a result, the memory subsystem is partially underutilized by a single kernel, allowing the other kernels with memory operations to overlap (as supported by the performance improvements reported in Section 7.3). Further, in our framework, we do not distinguish between compute-bound and memory-bound kernels during stream allocation. Rather than relying on speculative classification of kernels as compute-bound or memory-bound, which is challenging due to dependencies on data inputs, problem size, runtime configuration, and GPU architecture, our framework prioritizes inter-kernel

Repository	K_{tot}	K_{strm}	M_{tot}	M_{async}	Issues
ETHMINER [Ethminer 2017]	3	3	10	0	AP
Open3D [Zhou et al. 2018]	87	81	11	10	✗
Tengine [Alibaba 2012]	16	0	9	0	✗
NVIDIA CCCL [CCCL Team 2020]	194	8	64	17	AP
NVIDIA CUTLASS [Nvidia 2017]	255	74	63	21	✗
ONEFLOW [Oneflow 2017]	405	402	92	82	AP, MSB
ElasticFusion [Mp3guy 2015]	16	0	7	0	✗
cuML [Rapidsai-cuML 2018]	192	175	33	26	AP, MSB, ISA
DeepMD Kit [Wang et al. 2018]	46	0	44	0	AP, MSB
HeavyDB [Heavyai 2017]	12	2	15	6	✗

Fig. 20. Open source projects and corresponding statistics on manual stream allocation of kernels.

dependency analysis, ensuring correctness and architectural portability, while naturally exploiting compute and memory overlap whenever it arises.

Furthermore, our analysis correctly identified the absence of optimization opportunities without false positives in benchmark programs such as ‘`gemm`’, ‘`gesumv`’, ‘`adam`’, and ‘`adv`’ (not shown in the table), due to either the lack of multiple kernel launches or dependencies among unstreamed calls.

Next, we discuss the time taken by our static analysis phase (sixth column) and the time taken by the transformation pass (seventh column). The analysis time, T_{anlys}, includes the total time taken by the read-write analysis pass, CRA analysis, and DFSA algorithm. The analysis time does not depend on the size of the input program but on the complexity of the control flow, the number of unstreamed calls, and the complexity of the dependence graph. For example, the benchmark program, ‘`reaction`’, despite having a relatively lesser number of Lines of Code, has a significantly higher analysis time for its complex CFG (more nested loops and branches), a large number of unstreamed calls (more nodes in the dependence graph) and a complex relationship between the nodes (higher number of edges).

The successful identification of all optimization opportunities in PolyBench and HeCBench programs, coupled with analysis and transformation times mostly under 100 milliseconds (except for a few exceeding this time), demonstrates the effectiveness and practicality of StreamAlloc for real-world GPU compilers.

7.2 RQ2. Analysis of Manual Stream Allocation: Statistics and Corresponding Issues in Open-Source CUDA Projects

This section investigates whether real-world open-source projects use manual stream allocation to achieve parallel processing and the challenges that come with it, focusing on GitHub applications. Further, we discuss the sync2async optimization opportunities identified in these open-source projects in detail later in the section.

To conduct this study, we have surveyed several CUDA/C++ projects on GitHub and collected statistics from the top 10 most forked repositories. The ‘most forked’ GitHub filter shows popular projects with significant community contributions. The collected data is presented in Figure 20. In the table, the second column, K_{tot} represents the total count of CUDA kernel launches in each project, while K_{strm} signifies the subset of kernels in K_{tot} that are launched in non-default streams. Similarly, M_{tot} and M_{async} signify the total number of memcpy statements and the number of asynchronous memcpy statements in non-default streams. The findings reveal that while some projects employ non-default streams to maximize parallel processing, the majority still execute

kernels and memcpy statements on the default stream without computational overlap, restricting overall parallelism.

The last column represents different categories of stream-related issues in the corresponding GitHub projects. While some projects show no such issues (marked as ✗), the rest have reported various stream-related issues from open-source contributors. According to our observations, GitHub issues related to streams can be divided into three categories: (1) Async for Performance (AP), (2) Inefficient Stream Allocation (ISA), and (3) Misplaced Synchronization Barriers (MSB). **AP**: In these issues [Deepmodeling 2023; Ethereum-Mining 2018; NVIDIA-CCCL 2021; Oneflow-Inc 2021; Rapidsai 2023; Tensorflow 2023], users or developers suggest using asynchronous API calls instead of synchronous ones to improve performance. **ISA**: These issues [Rapidsai 2021, 2022] highlight incorrect stream allocation to unstreamed calls, potentially limiting overall parallelism. **MSB**: In these issues [Deepmodeling 2023; Oneflow-Inc 2018, 2022; Rapidsai 2020], users or developers suggest removal or placement of synchronization barriers (e.g. `cudaStreamSynchronize`) at optimal program points so as to ensure program's correctness and maximum parallelism. We present examples from each category of GitHub issues. In the DeepMD Kit project, one issue [Deepmodeling 2023] highlights user concerns regarding the use of default stream, recommending its replacement with non-default streams for improved performance (**AP**). Additionally, in the same issue, users emphasize the need for appropriate synchronization calls when using non-default streams (**MSB**). Another issue [Rapidsai 2021] from the cuML project reports that all GPU kernels in the Random Forest application are assigned to the same stream, causing false dependencies and resulting in a performance bottleneck (**ISA**).

The presence of stream-related issues reported by open-source contributors highlights the necessity, complexity, and error-prone nature of allocating streams to unstreamed calls. This clearly demonstrates the need for automatic support to handle stream allocation effectively.

Sync2async Optimization Opportunities. To analyze how sync2async opportunities arise in practice, we examine the open-source GPU projects (introduced earlier in this section). Projects such as CUTLASS and CCCL provide APIs that take a CUDA stream as an argument from the end library users, whereas others, such as DeepMD-kit, manage streams internally within the API function implementation. In the following discussion, we first describe the sync2async opportunities possible in CUTLASS and CCCL, followed by those identified in DeepMD-kit.

Projects, such as CUTLASS and CCCL, are implemented as libraries that expose host-side APIs that encapsulate GPU kernel launches and data transfer operations, abstracting low-level details to provide a simplified, user-friendly interface to end users. In such designs, API functions often accept a CUDA stream as an argument, binding all enclosed kernel launches and data transfers to the user-specified stream. For example, Figure 21a illustrates a snippet from the `Gemm` (General Matrix Matrix Multiplication) class of the CUTLASS repository, where the API function `run` (at Line 3) expects a stream as an argument from the end user, and launches the `Gemm` kernel (at Line 5) in the same user-specified stream. Although sync2async optimization opportunities may not be directly exposed in such cases, they manifest at the granularity of API invocations, where independent invocations issued by the end user can be scheduled on separate streams, enabling potential overlap among several operations across different invocations. Figure 21b illustrates a CUTLASS example where two independent GEMM operations are invoked on the GPU. Through the API, the first GEMM invocation multiplies tensors `A0` and `B0` to produce `D0`, and the second invocation (Line 21) multiplies `A0` and `B1` to produce `D1`. Both GEMM operations (Lines 19 and 21) invoke the overloaded `operator()` in the `Gemm` class (Line 10, Figure 21a), which in turn calls the `run` function internally (Lines 3, Figure 21a). Further, the code first transfers the input and pre-initialized output tensors from host to device (Lines 3–5) and, after the computations, retrieves

```
class Gemm {

 Status run(cudaStream_t stream =
      nullptr) {
  ...
  cutlass::Kernel<GemmKernel><<<
       grid, block, smem_size,
       stream>>>(params_);
  ...
 }


 Status operator()(Arguments const
       &args, void *workspace =
      nullptr, cudaStream_t stream
       = nullptr) {
   Status status = initialize(args
        , workspace, stream);

   if (status == Status::kSuccess)
         {
     status = run(stream);
   }
  return status;
 }

};
```

(a) CUTLASS GEMM API [Nvidia 2025b].

```
bool test(){
 ...
 tensor_A0.sync_device(); tensor_B0.sync_device();
 tensor_D0.sync_device(); tensor_B1.sync_device();
 tensor_D1.sync_device();
 ...
 typename Gemm0::Arguments arguments_0 {...,
    tensor_A0.device_ref(), tensor_B0.device_ref(),
             tensor_D0.device_ref(), ...};
 typename Gemm1::Arguments arguments_1{...,
    tensor_A0.device_ref(), tensor_B1.device_ref(),
    tensor_D1.device_ref(), ...};
 ...
 Gemm0 gemm_op_0;
 Gemm1 gemm_op_1;
 ...
 status = gemm_op_0.initialize(arguments_0, ...);
 status = gemm_op_1.initialize(arguments_1, ...);
 ...
 status = gemm_op_0();
 ...
 status = gemm_op_1();
 ...
 tensor_D0.sync_host();
 tensor_D1.sync_host();
 ...
}
```

(b) Code using CUTLASS GEMM APIs [Nvidia 2025a].

Fig. 21. Code snippet showing CUTLASS API codes and sync2async opportunity while using APIs.

the results via synchronous device-to-host transfers (Lines 23 and 24). As no user-defined streams are specified, all these operations are issued on the default stream, even though the API functions internally manage kernel launches and data transfers. This sequential execution prevents overlap across operations, including (1) host-to-device transfers of `B1` and `D1` (Lines 4 and 5) with the first GEMM computation (Line 19), (2) the two independent GEMM operations themselves (Lines 19 and 21); and (3) the second GEMM operation at Line 21 with the device-to-host transfer of the final result `D0` at Line 23 (assuming the first GEMM operation completes prior to the second GEMM operation). StreamAlloc can be extended to handle such API invocations without changing its core logic or algorithmic foundation. This enables StreamAlloc to assign appropriate streams to API calls, allowing the CUDA operations within the library API implementations to take advantage of asynchronous execution.

Projects, such as the DeepMD kit, implement library APIs without exposing CUDA streams to users as arguments. These API implementations further invoke device synchronization operation (see Line 5, Figure 22) at the beginning to prevent overlap with the CUDA operations launched by the driver/client code. This design choice allows StreamAlloc to assign streams to kernel launches and data transfer operations within the function based on dependencies. For example, in the `build_nlist_gpu` function (Figure 22), constructing the neighbour list of atoms involves multiple operations, some of which are independent and can overlap. The `build_nlist` kernel (Line 10) identifies valid (within a cutoff distance) neighbour indices, then a parallel prefix scan (Line 12) counts valid neighbours and computes compacted index positions, followed by the `fill_nlist` kernel (Line 14) which compacts the sparse list into a dense array, and finally a `cudaMemcpy` (Line 17) transfers the neighbour counts back to the host for downstream computations. Here, the data transfer operation and `fill_nlist` kernel are independent of each other, and

```
int build_nlist_gpu(...) {
 if (mem_size < nall) {
    return 1;
  }
 cudaDeviceSynchronize();
 ...
 int* numneigh = nlist.numneigh;
 int** firstneigh = nlist.firstneigh;
 ...
 build_nlist<<<...>>>(temp_nlist, ...);
 cudaDeviceSynchronize();
 parallel_prefix_scan<TPB><<<...>>>(numneigh, temp_nlist, ...);
 cudaDeviceSynchronize();
 fill_nlist<<<...>>>(firstneigh, temp_nlist, ...);
 cudaDeviceSynchronize();
 int* numneigh_host = new int[nloc];
 cudaMemcpy(numneigh_host, numneigh, ..., cudaMemcpyDeviceToHost);
 ...
}
```

Fig. 22. Code snippet from DeepMD kit project showing sync2async optimization opportunities.

can therefore be assigned to different streams, a sync2async optimization opportunity which our framework automatically identifies.

Our framework automatically detected and optimized four opportunities on multiple library functions of the DeepMD-kit project. A key challenge in measuring the performance impact of these optimizations was the absence of comprehensive test environments for several DeepMD-kit API functions, which required us to develop dedicated test cases for all functions with identified sync2async opportunities. We generated these test cases using random inputs designed to capture a broad range of realistic execution scenarios. In the optimized functions, we observed performance gains ranging from 3% to 8%. As these functions are part of an already highly optimized production codebase with limited overlap opportunities, the observed performance gains (although modest) highlight our framework's effectiveness in uncovering additional overlap opportunities in real-world GPU applications.

7.3 RQ3. Speedup due to sync2async Optimization

Figures 23 and 24 show the performance improvement obtained by applying sync2async to several PolyBench and HeCBench benchmark programs (Section 7.1) with optimization opportunities on P100, A4000, and A100 machines. We run all benchmark programs five times before and after transformation, then report their arithmetic mean as execution times. StreamAlloc assigns stream identifiers for each unstreamed call and inserts stream synchronizations at optimal points to maximize parallelism through asynchronous execution while preserving dependencies. Additionally, it removes the explicit barrier synchronization calls (e.g., `cudaDeviceSynchronize` and `cudaThreadSynchronize`) from the given input programs. As shown in Figures 23 and 24, sync2async optimization results in significant performance gain on the P100, A4000 and A100 GPUs with the average (geomean) speedups of 1.49x, 1.63x and 2.02x (up to 2.8x, 3.7x and 4.3x speedups, respectively).

The transformed code will yield the best possible result if the overlapped computations and data transfers can completely hide each other's latency. In several programs such as '`corr`', '`covar`', '`jacobi1d`' from PolyBench, and '`gaussian`', '`rsmt`' from HeCBench, the opportunity for **KK** overlap is not present, but **KM** overlap is possible. In `jacobi1d`, the device-side computation time is small compared to that of the data transfer time, and hence, the performance gain due to the overlap is modest.

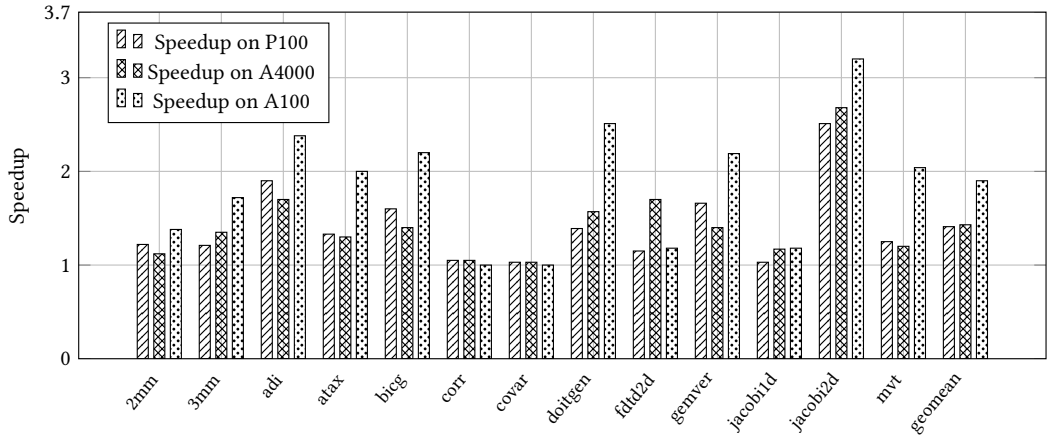


Fig. 23. Speedup achieved by sync2async over unstreamed PolyBench on P100, A4000 and A100 GPUs.

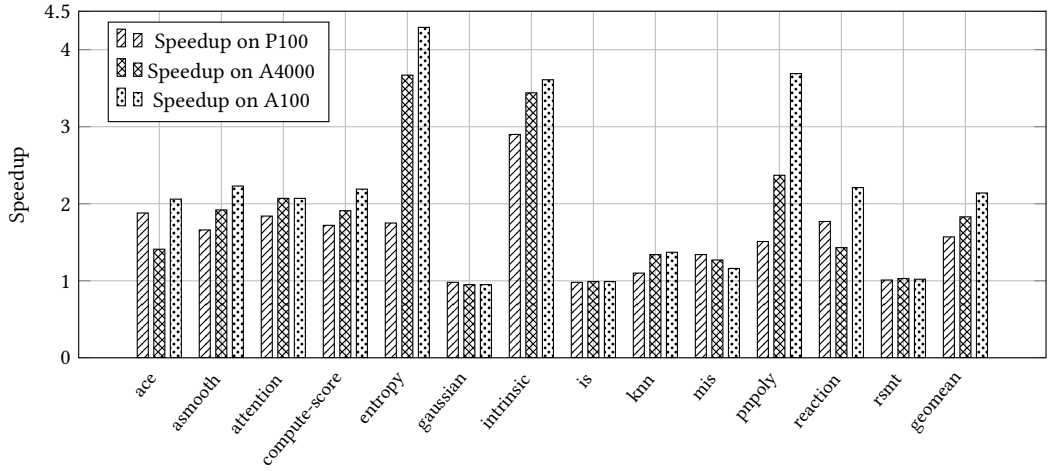


Fig. 24. Speedup achieved by sync2async over unstreamed HeCBench on P100, A4000 and A100 GPUs.

In contrast, for ‘`covar`’, ‘`corr`’, ‘`gaussian`’, and ‘`rsmt`’, the device-side computation time is significantly higher than the data transfer time. For instance, in ‘`gaussian`’, the computation time is nearly 12 times more than the time taken by data transfer operations. Consequently, despite applying the optimization, the overall execution time remains unchanged compared to the original program, as the prolonged device-side computation overshadows any benefits obtained through overlap. The ‘`is`’ benchmark from HeCBench exhibits opportunities for both **KK** and **KM** overlap. However, in **KK**, overlap is limited to a subset of kernel launches, while the remaining kernels have high computation times that dominate overall runtime. Further, in **KM** overlap, kernel execution remains much longer than data transfer, yielding negligible performance benefits from overlap. Further, note that the speedups achieved on the A4000 and A100 machines are much higher than those of the P100 machines. It is interesting to note that the speed-up depends not only on the overlap but also on the GPU hardware characteristics. A4000 and A100 machines have more computing

Benchmark	O_{manual}	O_{auto}
vec	✓	✓
b&s	✓	✗
image	✓	✓
ml	✓	✓
hits	✓	✓
dl	✓	✓

(a) Opportunities in GrCUDA.

Suite	Benchmark	$\#S_a$	$\#S_g$	$\#L_o$	$\#L_g$
PolyBench	2mm	7	5	37	67
	3mm	10	7	45	100
	adi	11	4	46	97
	atax	6	4	32	70
	bicg	7	5	39	82
	correlation	8	5	44	101
	covarinace	6	3	33	92
	doitgen	6	4	32	77
	fdtd2d	8	5	33	70
PolyBench	gemver	12	9	58	114
	jacobi1d	6	3	27	58
	jacobi2d	6	3	27	58
	mvt	7	5	35	68
GrCUDA	vec	3	2	12	26
	image	7	5	27	66
	ml	5	2	28	61
	hits	3	2	28	54
	dl	3	2	24	55

(b) Comparison of stream counts and relative code-size overhead.

Fig. 25. Comparison between StreamAlloc and CUDA Graph.

cores and higher PCI-Express bus bandwidth than P100, enabling greater parallelization through concurrent kernel execution and achieving higher speedups through data transfer-computation overlap.

7.4 RQ4. Comparison of StreamAlloc with CUDA Graph and Manual Stream Scheduling

In this section, our aim is to assess whether the efficacy of StreamAlloc can be comparable to that of manual stream scheduling and whether stream allocation through StreamAlloc can deliver performance comparable to CUDA Graph. For this experiment, we have evaluated our framework on six benchmarks part of an open-source benchmark suite, GrCUDA [Parravicini et al. 2021], and have also compared against the manually transformed CUDA Graph variants of the PolyBench benchmark programs. Figure 25a illustrates the effectiveness of StreamAlloc in relation to manual stream allocation for the programs in the GrCUDA benchmark suite. The programs are available in *sync*, *async*, and *CUDA Graph* versions. Each *sync* CUDA program has been manually transformed (by Parravicini et al. [2021]) into its corresponding *async* version. In Figure 25a, the second column, O_{manual}, indicates the optimization opportunities found by the manual stream allocation process, whereas O_{auto} shows the automatically found optimization opportunities by our framework. Note that our framework can automatically identify sync2async optimization opportunities in nearly all benchmark programs (except `b&s`), indicating that its effectiveness matches that of the manual process. The `b&s` benchmark (shown in Figure 26a) has two data transfer operations (Lines 2 and 3) and one device kernel launch (Line 5). The data transfers and the kernel execution happen over multiple iterations in a loop over different data points of a single array. Hence, to achieve **KK** overlap, the kernel in each iteration should be launched in a separate stream (as shown in Figure 26b). Similarly, **KM** overlap can also be exploited, where the device kernel execution in one loop iteration can overlap with the data transfer operations initiated in the subsequent iteration. However, due to the conservative behaviour, our framework cannot determine if the data transfers or the kernel execution is invoked over various iterations on the same data array. Consequently, the analysis spuriously reports the presence of dependencies across iterations, leading to a loss of opportunity. Such opportunities are uncommon in real-world programs, and addressing these cases would require a precise range analysis, which is complex to implement. To maintain the practicality and conciseness of our framework, we have opted not to include such a complex analysis.

CUDA Graph is a feature in NVIDIA's CUDA programming model that allows programmers to create a DAG of interdependent computations that can be scheduled for asynchronous execution. To evaluate how closely our framework approximates the optimal configuration for maximal overlap, we compare the number of streams automatically generated by our framework ($\#S_a$) with those

```
for (int j = 0; j < M; j++) {
 cudaMemPrefetchAsync(x[j], ..., 0);
 cudaMemPrefetchAsync(y[j], ..., 0);
 ...
 bs<<<num_blocks, block_size_1d>>>(x[j
      ], y[j], N, R, V, T, K);
 cudaDeviceSynchronize();
}
```

(a) with default stream

```
for (int j = 0; j < M; j++) {
 cudaMemPrefetchAsync(x[j], ..., s[j]);
 cudaMemPrefetchAsync(y[j], ..., s[j]);
 ...
 bs<<<num_blocks, block_size_1d, 0,
      s[j]>>>(x[j], y[j], N, R, V, T, K
      );
}
```

(b) with non-default stream

```
if (iter == 0) {
 cudaGraphCreate(&graphs[0], 0);
 for (int j = 0; j < M; j++) {
    void *kernel_args[7] = {(void *)&x[j], (void
         *)&y[j], &N, &R, &V, &T, &K};

    dim3 tb(block_size_1d);
    dim3 b_size(num_blocks);

    add_node(kernel_args, kernel_params[j], (void
         *)bs, b_size, tb, graphs[0], &kernels[j
         ], nodeDependencies);
 }
 cudaGraphInstantiate(&graphExec[0], graphs[0],
      NULL, NULL, 0);
}

cudaGraphLaunch(graphExec[0], s[0]);
```

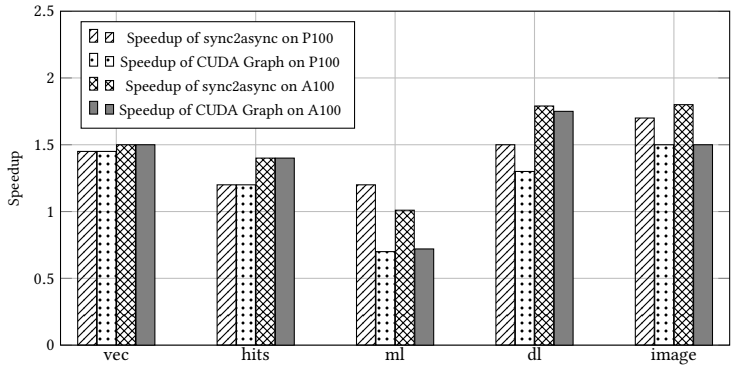
(c) with CUDA Graph

Fig. 26. b&s application with default & non-default streams and with CUDA Graph.

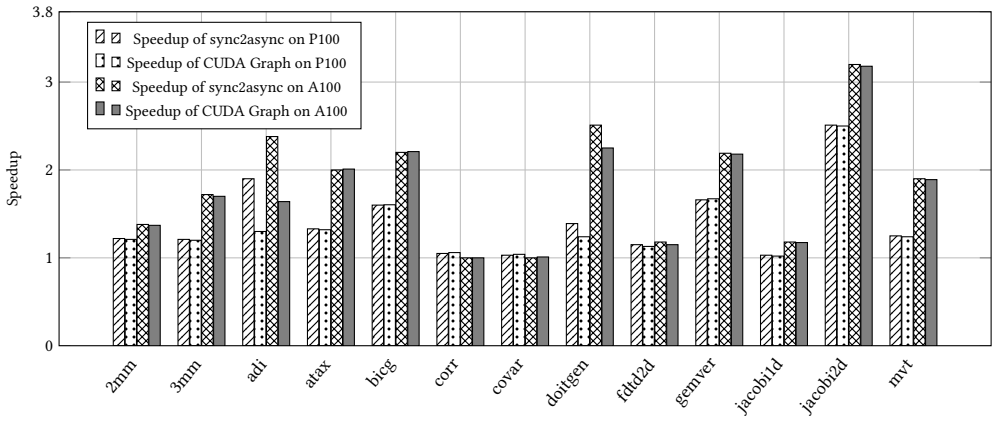
observed in the corresponding manually[4] constructed CUDA Graph variants ($\#S_g$), obtained from the NVIDIA Nsight profiling. As shown in Figure 25b, in most benchmark programs, $\#S_a$ is slightly higher (with an average difference of 2.5) than $\#S_g$, reflecting the difference between the static, conservative stream scheduling approach (DFSA) of our framework and the dynamic scheduling approach of CUDA Graph, where the runtime assigns operations to streams from a shared pool based on programmer-specified dependencies. We believe that in the CUDA Graph approach, the runtime opportunistically reuses streams that become idle, thereby minimizing the total number of streams needed. Despite this, performance comparisons (Figures 27a and 27b, discussed later in this section) show that our framework performs on par with or better than CUDA Graph.

Manually identifying dependencies and creating graphs using CUDA Graph APIs can be complex for programmers and requires a deep understanding of the CUDA programming model, which can increase the development time. For instance, in Figure 26c, the implementation of the b&s application via the CUDA Graph framework showcases the elevated complexity in development compared to the straightforward *async* version of the same program in Figure 26b. To further validate these observations, Figure 25b compares the code size of CUDA Graph implementations ($\#L_g$) with their original (unstreamed) counterparts ($\#L_o$). In the original programs, we count only the lines responsible for GPU kernel launches and data transfer operations (i.e., cudaMemcpy and kernel launch statements). For the CUDA Graph implementations, we include these lines performing kernel launches and data transfers, as well as all additional statements required for explicit dependency specification (for graph creation), graph object instantiation, and graph execution. Across all benchmark programs (from PolyBench and GrCUDA), the CUDA Graph versions contain, on average, 116.3% more lines of code, reflecting the additional programming complexity introduced by manual dependency specification and graph construction. This indicates that adapting the existing large and complex code bases to use CUDA Graph may require significant modifications, which is time-consuming and error-prone. In contrast, our framework leverages compile-time analysis and automated transformation passes, performing code injection and modification directly at the

[4]While GrCUDA is readily available, we converted the PolyBench programs to CUDA Graph implementations through manual dependency analysis and graph construction, which took approximately 1.5 weeks for the full benchmark suite.



(a) Speedup comparison of sync2async and CUDA Graph on GrCUDA programs.



(b) Speedup comparison of sync2async and CUDA Graph on PolyBench programs.

Fig. 27. Speedup comprison of sync2async and CUDA Graph.

intermediate representation (IR) level, thereby eliminating the need for programmers' intervention at the source level.

Performance against CUDA Graphs. Unlike traditional kernel launches, the creation of CUDA Graphs at runtime will add significant overhead. Figures 27a and 27b show the speedups (relative to the default stream) achieved by the sync2async and the CUDA Graph versions on the GrCUDA and PolyBench benchmark suites. In some benchmarks, sync2async achieves higher speedups than CUDA Graph variants due to the graph instantiation overhead in CUDA Graph programs. This cost becomes significant in the case of large and structurally complex graphs where individual nodes have multiple child nodes due to the source program's intricate control flow and data dependencies. For example, the `adi` benchmark involves multiple device kernel launches within several nested `for` loops, resulting in an overall instantiation cost of approximately 40 ms in the CUDA Graph implementation. Notably, despite the additional overhead of CUDA Graph, sync2async achieves modest performance gains over CUDA Graph across most benchmarks. This behaviour is primarily due to two factors. First, in benchmarks with relatively simple control flow and data dependencies, the CUDA Graphs are structurally simple, thereby incurring negligible instantiation costs. For example, the `2mm` benchmark has single-child dependencies, which yield a simple graph structure with an instantiation cost of only 70 µs. Second, in benchmarks where kernel execution and data

transfer times dominate the overall runtime, the instantiation overhead becomes negligible in comparison. For instance, in `fdtd2d`, instantiation takes 7 ms compared to approximately 400 ms for combined kernel and data transfer execution.

Performance against Manual Optimization. We evaluated our framework against manually optimized benchmark programs from PolyBench and GrCUDA benchmark suites. Across most benchmarks, the performance differences between manual and automatic sync2async optimizations are negligible (with a median difference of approximately 0.5%). However, in the '`adi`' benchmark, the nested loop and complex dependency graph led our conservative DFSA algorithm to allocate more streams than the hand-tuned schedule. While the host-side stream creation overhead is negligible, repeated synchronization and stream-switching costs in a loop accumulate, producing a performance difference of approximately 21% on GPUs such as P100 and A4000; on more advanced hardware like the A100, this difference decreased to around 8%. Further, Nsight profiling confirms that the overlap achieved by our automatic approach is identical to that of manually optimized schedules. Our experience with manual optimization underscores the challenges involved. Starting from a naive stream-scheduled implementation, we iteratively tuned each benchmark to reduce the number of streams. While we successfully lowered the counts to match those used in CUDA Graphs, further reduction was infeasible without sacrificing opportunities for overlap. The process required approximately 3–4 days of careful analysis to understand dependencies, determine appropriate stream allocations, and correctly place synchronization operations. This demonstrates that manual stream scheduling demands significant expertise, careful tuning, and considerable time, and remains prone to errors. In contrast, our framework, though conservative, consistently delivers near-optimal performance with correctness, providing a practical and reliable alternative to manual stream management.

Overall, these results demonstrate that sync2async is competitive with both CUDA Graph and expert-crafted manual implementations, offering more practicality and portability. We believe this establishes automatic stream allocation as a robust and scalable alternative for both existing and emerging GPU workloads.

8 DISCUSSION

Alias Analysis. Our framework uses an intra-procedural alias analysis, which assumes that all pointer parameters of a function are potential aliases, unless the programmer explicitly annotates them with the `__restrict` keyword. Further, the conservative alias analysis cannot differentiate between two different memory segments of the same data array (discussed in Section 7.4). These design choices can restrict optimization opportunities. Future enhancements could incorporate more precise techniques, such as context-sensitive inter-procedural analysis [Emami et al. 1994; Whaley and Rinard 1999; Wilson 1998] or range analysis [Harrison 1977], to identify additional opportunities, albeit with increased analysis complexity and time overheads.

Dynamic Parallelism. StreamAlloc currently treats GPU functions as atomic execution units, and therefore, it does not assign streams to child kernels in the context of CUDA Dynamic Parallelism (CDP). Additionally, our framework currently handles synchronization entirely on the host side, using `cudaStreamSynchronize` to enforce explicit dependency ordering. Such synchronization primitives are unavailable in the context of CDP, and it relies on device-side mechanisms such as `cudaStreamTailLaunch`, which enqueues a child kernel to execute only after all operations in the parent kernel have completed. This semantic difference necessitates a distinct analysis model tailored specifically for device-level dependency ordering. Further, there is a critical performance trade-off inherent in stream creation on the GPU. For example, if a parent kernel launches child kernels from multiple threads, naively creating and managing per-thread streams can introduce

substantial overhead, potentially outweighing any overlapping benefits. Efficient CDP-aware stream management would therefore require a carefully designed cost model and potentially batched stream allocation, constituting a future research problem.

Instruction Reordering. Figure 1b illustrates a scenario in which the execution of `mvt_kernel1` (Line 10) can potentially overlap with data transfers scheduled on streams `s3` and `s5` (Lines 5 and 7), depending on the runtime scheduling order of transfers on `s3` (Line 5) and `s4` (Line 6) on the copy engine. An interesting case arises if the instructions were reordered, swapping the data transfers on `s3` and `s4` (swapping Lines 5 and 6), followed by moving the `mvt_kernel1` invocation along with its associated stream synchronization before the transfers on `s3` and `s5` (moving Lines 9 and 10 before Line 6). Such a reordering can increase the chances of overlap between `mvt_kernel1` and the data transfer on `s3`. However, statically determining profitable instruction reordering that preserves program correctness and maximizes overlap is challenging. While these transformations could yield additional performance benefits beyond the overlap opportunities already captured by StreamAlloc, they lie outside the primary focus of this work and can be implemented as an independent post-processing optimization.

9 RELATED WORK

Overlapping Memory Copies and Computations. HUM (Hidden Unified Memory) [Jung et al. 2020] is a runtime technique that uses Unified Memory [CUDA-UVM 2025] to execute host-to-device memcpy commands asynchronously by enabling the overlap of host-to-device page transfer operation with host computations or kernel computations. It exploits the operating system's access pattern of pages to handle data consistency at runtime. StreamAlloc differs from HUM as: (1) it relies on a static analysis technique rather than changing the complex runtime, (2) it can also overlap multiple kernel computations with each other and memory copies (host-to-device and device-to-host) with each other rather than only overlapping host-to-device memcpy time with host/kernel computations as HUM, and (3) it avoids making memcpy and kernel calls overlap if they have dependencies, whereas HUM handles data consistency on demand at runtime.

Parravicini et al. [2021] provided a new GPU runtime scheduler that infers data dependencies between GPU kernels by modelling them as Directed Acyclic Graph and schedules them to maximize data transfer-computation overlap and space-sharing (achieved when running multiple concurrent GPU kernels). This runtime-based approach creates a partial dependence graph, unlike StreamAlloc, which builds a complete dependence graph through static code analysis. Moreover, while this approach necessitates that data structures reside in unified memory to attain transfer-computation overlap, we achieve it by converting synchronous data transfers into asynchronous ones using CUDA streams.

GSOHC [Kumar Basu and Vedurada 2025] provides hetero-sync motion optimization which relocates poorly placed synchronization barriers and synchronous data transfers using context-sensitive data-flow analysis to enable overlap between host and device computations. However, it does not exploit the overlap among device computations or the overlap between device computations and data transfers. In contrast, StreamAlloc achieves these overlaps also by transforming synchronous data transfers into asynchronous transfers, and automatically assigning kernel launches and transfers to appropriate non-default streams.

Optimal Co-scheduling of GPU Kernels. Kim et al. [2016] proposed a software-hardware solution for parallelizing interdependent GPU kernels by profiling data dependencies (for sampling-based memory access range analysis) at runtime and employing per-page reference counting. Similarly, the runtime approach [Shekofteh et al. 2020] aims at co-scheduling of GPU kernels by breaking large kernels into sub-kernels, profiling and ranking them based on execution behaviour,

and overlapping thread blocks to maximize GPU resource utilization (the underlying assumption is the absence of data dependencies between kernels). Likewise, Liang et al. [2015] proposes a software-hardware solution that uses hardware leaky-bucket-based thread-block interleaving to enable kernels from multiple applications to share resources spatially and temporally. Beheshti Roui et al. [2020] introduces a framework that finds the pairs of kernels to be streamed (stream set) and the order of streamed programs using a performance model and scheduler and employs CUDA streams for parallel kernel execution.

Cruz et al. [2019] optimizes GPU kernel submission order on streams to maximize resource utilization by modelling the optimization as a series of knapsack problems with kernel weights and values based on resource use and execution time estimation (obtained through NVIDIA profiler tools). Similarly, Eyraud-Dubois and Bentes [2020] introduces a preemptive co-scheduling algorithm for concurrent GPU kernel execution, striving to minimize preemptions and optimize makespan of kernel launches through graph-based strategies and Linear Programming. PySchedCL [Ghose et al. 2022] is a framework that accepts the specification of tasks in the form of an OpenCL DAG and efficiently schedules kernels by exploring fine-grained concurrency-aware strategies and offers customizable scheduling algorithm support. Unlike StreamAlloc, many of these works require programmers to specify multiple tasks as a DAG to determine the most effective scheduling.

Static Analyses to Minimize Memory Copy Overhead. Ashcraft et al. [2019] uses a whole-program analysis to identify data that needs to be transferred between CPU and GPU, and perform liveness and dominator tree analyses to determine the optimal placement (i.e. scheduling) of the data transfers so that the total amount of data transferred is minimized. Similarly, Barua et al. [2020] detects and eliminates redundant data movements by formulating the problem as a partial redundancy elimination (PRE) problem and applying the lazy code motion. Kessler [2019] proposes a static analysis technique to reorder operand arrays in memory by analyzing kernel-operand dependence graphs, so that data transfer startup overheads and memory allocation overheads can be reduced by fusing transfers and memory allocations of multiple operands adjacent in memory. Since these existing static analysis techniques lack the utilization of streams to optimize data transfer and communication overlap, StreamAlloc cannot leverage them, necessitating the development of a new static analysis technique, as proposed in the paper.

10 CONCLUSION AND FUTURE WORK

We have developed a framework StreamAlloc that statically analyzes CPU-GPU heterogeneous programs to identify opportunities for sync2async optimization, which assigns appropriate stream identifiers for unstreamed calls. Further, it also finds out the appropriate locations for stream synchronization calls in a given input program. Our static analysis framework combines a compositional read-write analysis and a flow-sensitive data-flow CRA analysis to identify parallelizable unstreamed calls and a scheduler to allocate non-default streams. This enables automatic code transformation, allowing GPU computation, memory transfers, and host-side computation to overlap while ensuring correctness. We have also evaluated our framework on benchmark programs from the well-known benchmark suites, PolyBench and HeCBench. Our evaluation shows that our StreamAlloc effectively identifies sync2async optimization opportunities in the chosen benchmark programs and achieves an overall geometric mean speedup of 1.7x on P100, A4000, and A100 GPUs, compared to the baseline code.

Our framework, StreamAlloc, is designed for single-GPU systems, and extending it to multi-GPU setups is a natural next step that introduces additional considerations. Since streams are local to each GPU context, extending to multi-GPU requires independent stream management for each context. Further, cross-GPU data dependencies must be carefully tracked, as computations on one GPU may rely on results from another. Maintaining correctness in this case requires careful

placement of stream synchronization operations within the appropriate device context. Additionally, coordinating operations across contexts necessitates event-based synchronizations, which must be explicitly managed. Addressing these factors will enable efficient asynchronous execution across multiple devices. Another direction involves generalizing our approach to other CPU-GPU programming models, such as OpenCL and SYCL, and heterogeneous computing architectures such as FPGAs. In OpenCL and SYCL, kernels can be dispatched to multiple devices, such as CPUs, GPUs, or FPGAs, each with its own command queues, memory hierarchies, and synchronization semantics. Future work can focus on automatically analyzing dependencies across these devices and mapping them to the appropriate queues, enabling efficient asynchronous execution.

REFERENCES

Alibaba. 2012. GitHub - alibaba/tengine: A distribution of Nginx with some advanced features. https://github.com/alibaba/tengine

Gabriell Alves de Araujo, Dalvan Griebler, Marco Danelutto, and Luiz Gustavo Fernandes. 2020. Efficient NAS Parallel Benchmark Kernels with CUDA. In *2020 28th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. nill, nill, 9–16. https://doi.org/10.1109/PDP50117.2020.00009

Matthew B Ashcraft, Alexander Lemon, David A Penry, and Quinn Snell. 2019. Compiler optimization of accelerator data transfers. *International Journal of Parallel Programming* 47, 1 (2019), 39–58.

Michal Babej and Pekka Jääskeläinen. 2020. HIPCL: Tool for Porting CUDA Applications to Advanced OpenCL Platforms Through HIP. In *Proceedings of the International Workshop on OpenCL* (Munich, Germany) *(IWOCL '20)*. Association for Computing Machinery, New York, NY, USA, Article 18, 3 pages. https://doi.org/10.1145/3388333.3388641

Seonmyeong Bak, Oscar Hernandez, Mark Gates, Piotr Luszczek, and Vivek Sarkar. 2021. Task-Graph Scheduling Extensions for Efficient Synchronization and Communication. In *Proceedings of the ACM International Conference on Supercomputing* (Virtual Event, USA) *(ICS '21)*. Association for Computing Machinery, New York, NY, USA, 88–101. https://doi.org/10.1145/3447818.3461616

Prithayan Barua, Jisheng Zhao, and Vivek Sarkar. 2020. OmpMemOpt: Optimized Memory Movement for Heterogeneous Computing. In *Euro-Par 2020: Parallel Processing*, Maciej Malawski and Krzysztof Rzadca (Eds.). Springer International Publishing, Cham, 200–216.

Mohamad Beheshti Roui, S. Kazem Shekofteh, Hamid Noori, and Ahad Harati. 2020. Efficient Scheduling of Streams on GPGPUs. *J. Supercomput.* 76, 11 (nov 2020), 9270–9302. https://doi.org/10.1007/s11227-020-03209-x

CCCL Team. 2020. CCCL: CUDA C++ Core Libraries. https://github.com/NVIDIA/cccl

Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, nill, 44–54. https://doi.org/10.1109/IISWC.2009.5306797

Rommel A.Q. Cruz, Cristiana Bentes, Bernardo Breder, Eduardo Vasconcellos, Esteban Clua, Pablo M.C. de Carvalho, and Lúcia M.A. Drummond. 2019. Maximizing the GPU resource usage by reordering concurrent kernels submission. *Concurrency and Computation: Practice and Experience* 31, 18 (2019), e4409. https://doi.org/10.1002/cpe.4409 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/cpe.4409 e4409 cpe.4409.

CUDA. 2025. CUDA Programing Model. https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf.

CUDA-UVM. 2025. CUDA C Programming Guide - Unified Memory. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html. Accessed: 2025-03-09.

CudaGraphAPI. 2023. CUDA Graph. https://docs.nvidia.com/cuda/cuda-runtime-api/group__CUDART__GRAPH.html. Retrieved: 2023-08-08.

L. Dagum and R. Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE Computational Science and Engineering* 5, 1 (1998), 46–55. https://doi.org/10.1109/99.660313

Deepmodeling. 2023. Pull Request #2661 · DeepMd Kit. https://github.com/deepmodeling/deepmd-kit/pull/2661

Maryam Emami, Rakesh Ghiya, and Laurie J. Hendren. 1994. Context-sensitive interprocedural points-to analysis in the presence of function pointers. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation* (Orlando, Florida, USA) *(PLDI '94)*. Association for Computing Machinery, New York, NY, USA, 242–256. https://doi.org/10.1145/178243.178264

Ethereum-Mining. 2018. Pull Request #1345 · ethereum-mining/ethminer. https://github.com/ethereum-mining/ethminer/pull/1345

Ethminer. 2017. GitHub - Ethereum-mining-ethminer: Ethereum miner with OpenCL, CUDA and stratum support. https://github.com/ethereum-mining/ethminer

Lionel Eyraud-Dubois and Cristiana Bentes. 2020. Algorithms for Preemptive Co-scheduling of Kernels on GPUs. In *2020 IEEE 27th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. nill, nill, 192–201. https://doi.org/10.1109/HiPC50609.2020.00033

Xiaokang Fan, Yulei Sui, Xiangke Liao, and Jingling Xue. 2017. Boosting the precision of virtual call integrity protection with partial pointer analysis for C++. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis* (Santa Barbara, CA, USA) *(ISSTA 2017)*. Association for Computing Machinery, New York, NY, USA, 329–340. https://doi.org/10.1145/3092703.3092729

Anirban Ghose, Siddharth Singh, Vivek Kulaharia, Lokesh Dokara, Srijeeta Maity, and Soumyajit Dey. 2022. PySchedCL: Leveraging Concurrency in Heterogeneous Data-Parallel Systems. *IEEE Trans. Comput.* 71, 9 (2022), 2234–2247. https://doi.org/10.1109/TC.2021.3125792

Scott Grauer-Gray, Lifan Xu, Robert Searles, Sudhee Ayalasomayajula, and John Cavazos. 2012. Auto-tuning a high-level language targeted to GPU codes. In *2012 Innovative Parallel Computing (InPar)*. nill, nill, 1–10. https://doi.org/10.1109/InPar.2012.6339595

Yi Guo, Rajkishore Barik, Raghavan Raman, and Vivek Sarkar. 2009. Work-first and help-first scheduling policies for async-finish task parallelism. In *2009 IEEE International Symposium on Parallel & Distributed Processing*. IEEE, Rome, 1–12. https://doi.org/10.1109/IPDPS.2009.5161079

Yi Guo, Jisheng Zhao, Vincent Cave, and Vivek Sarkar. 2010. SLAW: A Scalable Locality-Aware Adaptive Work-Stealing Scheduler for Multi-Core Systems. *SIGPLAN Not.* 45, 5 (jan 2010), 341–342. https://doi.org/10.1145/1837853.1693504

W.H. Harrison. 1977. Compiler Analysis of the Value Ranges for Variables. *IEEE Transactions on Software Engineering* SE-3, 3 (1977), 243–250. https://doi.org/10.1109/TSE.1977.231133

Heavyai. 2017. GitHub - heavyai/heavydb: HeavyDB (formerly OmniSciDB). https://github.com/heavyai/heavydb

Tsung-Wei Huang, Dian-Lun Lin, Chun-Xun Lin, and Yibo Lin. 2022. Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System. *IEEE Transactions on Parallel and Distributed Systems* 33, 6 (2022), 1303–1320. https://doi.org/10.1109/TPDS.2021.3104255

Zheming Jin and Jeffrey S. Vetter. 2023. A Benchmark Suite for Improving Performance Portability of the SYCL Programming Model. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, nill, 325–327. https://doi.org/10.1109/ISPASS57527.2023.00041

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

Jaehoon Jung, Daeyoung Park, Youngdong Do, Jungho Park, and Jaejin Lee. 2020. Overlapping Host-to-Device Copy and Computation Using Hidden Unified Memory. In *Proceedings of the 25th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming* (San Diego, California) *(PPoPP '20)*. Association for Computing Machinery, New York, NY, USA, 321–335. https://doi.org/10.1145/3332466.3374531

Christoph Kessler. 2019. Global optimization of operand transfer fusion in heterogeneous computing. In *Proceedings of the 22nd International Workshop on Software and Compilers for Embedded Systems*. nill, nill, 49–58.

Khronos. 2020. https://registry.khronos.org/SYCL/specs/

Gwangsun Kim, Jiyun Jeong, John Kim, and Mark Stephenson. 2016. Automatically exploiting implicit pipeline parallelism from multiple dependent kernels for gpus. In *Proceedings of the 2016 International Conference on Parallel Architectures and Compilation*. nill, nill, 341–352.

Jakub Kuderski, Jorge A. Navas, and Arie Gurfinkel. 2019. Unification-based Pointer Analysis without Oversharing. In *2019 Formal Methods in Computer Aided Design (FMCAD)*. Springer, San Jose, California, USA, 37–45. https://doi.org/10.23919/FMCAD.2019.8894275

Soumik Kumar Basu and Jyothi Vedurada. 2025. GSOHC: Global Synchronization Optimization in Heterogeneous Computing. In *39th European Conference on Object-Oriented Programming (ECOOP 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 333)*, Jonathan Aldrich and Alexandra Silva (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 21:1–21:30. https://doi.org/10.4230/LIPIcs.ECOOP.2025.21

Chris Lattner and Vikram Adve. 2004. LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation. In *Proceedings of the International Symposium on Code Generation and Optimization: Feedback-Directed and Runtime Optimization* (Palo Alto, California) *(CGO '04)*. IEEE Computer Society, USA, 75.

Yuxiang Lei and Yulei Sui. 2019. Fast and Precise Handling of Positive Weight Cycles for Field-Sensitive Pointer Analysis. In *Static Analysis: 26th International Symposium, SAS 2019, Porto, Portugal, October 8–11, 2019, Proceedings* (Porto, Portugal). Springer-Verlag, Berlin, Heidelberg, 27–47. https://doi.org/10.1007/978-3-030-32304-2_3

Yun Liang, Huynh Phung Huynh, Kyle Rupnow, Rick Siow Mong Goh, and Deming Chen. 2015. Efficient GPU Spatial-Temporal Multitasking. *IEEE Transactions on Parallel and Distributed Systems* 26, 3 (2015), 748–760. https://doi.org/10.1109/TPDS.2014.2313342

LLVM. 2022. LLVM Clang. https://clang.llvm.org/. Retrieved: 2022-08-08.

Wenjie Ma, Shengyuan Yang, Tian Tan, Xiaoxing Ma, Chang Xu, and Yue Li. 2023. Context Sensitivity without Contexts: A Cut-Shortcut Approach to Fast and Precise Pointer Analysis. *Proc. ACM Program. Lang.* 7, PLDI, Article 128 (June 2023), 26 pages. https://doi.org/10.1145/3591242

Jordyn Maglalang, Sriram Krishnamoorthy, and Kunal Agrawal. 2017. Locality-Aware Dynamic Task Graph Scheduling. In *2017 46th International Conference on Parallel Processing (ICPP)*. IEEE, Bristol, UK, 70–80. https://doi.org/10.1109/ICPP.2017.16

Alberto Marchetti-Spaccamela, Nicole Megow, Jens Schlöter, Martin Skutella, and Leen Stougie. 2020. On the Complexity of Conditional DAG Scheduling in Multiprocessor Systems. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, Louisiana, 1061–1070. https://doi.org/10.1109/IPDPS47924.2020.00112

Sparsh Mittal and Shraiysh Vaishay. 2019. A survey of techniques for optimizing deep learning on GPUs. *Journal of Systems Architecture* 99 (2019), 101635. https://doi.org/10.1016/j.sysarc.2019.101635

Mp3guy. 2015. GitHub - mp3guy/ElasticFusion: Real-time dense visual SLAM system. https://github.com/mp3guy/ElasticFusion

Steven Muchnick et al. 1997. *Advanced compiler design implementation.* Morgan kaufmann, California.

NVIDIA. 2005. CUDA Toolkit Documentation. https://docs.nvidia.com/cuda/cuda-runtime-api/api-sync-behavior.html

Nvidia. 2017. GitHub - NVIDIA/cutlass: CUDA Templates for Linear Algebra Subroutines. https://github.com/NVIDIA/cutlass

NVIDIA. 2024. https://developer.nvidia.com/nsight-systems

Nvidia. 2025a. Dual GEMM Example in CUTLASS. https://github.com/NVIDIA/cutlass/tree/main/examples/45_dual_gemm

Nvidia. 2025b. Gemm Class in CUTLASS. https://github.com/NVIDIA/cutlass/blob/c6aeb9179c5f74a0fcdbd28527bf4b6ba8c60752/include/cutlass/gemm/device/gemm.h

NVIDIA-CCCL. 2021. Issue #768 · NVIDIA/cccl. https://github.com/NVIDIA/cccl/issues/768

Oneflow. 2017. GitHub - Oneflow-Inc/oneflow: OneFlow is a deep learning framework designed to be user-friendly, scalable and efficient. https://github.com/Oneflow-Inc/oneflow

Oneflow-Inc. 2018. Pull Request #890 · Oneflow-Inc/oneflow. https://github.com/Oneflow-Inc/oneflow/pull/890

Oneflow-Inc. 2021. Pull Request #7022 · Oneflow-Inc/oneflow. https://github.com/Oneflow-Inc/oneflow/pull/7022

Oneflow-Inc. 2022. Pull Request #8571 · Oneflow-Inc/oneflow. https://github.com/Oneflow-Inc/oneflow/pull/8571

OpenAI. 2023. ChatGPT: Generative Pre-trained Transformer. https://openai.com/chatgpt

A. Parravicini, A. Delamare, M. Arnaboldi, and M. D. Santambrogio. 2021. DAG-based Scheduling with Resource Sharing for Multi-task Applications in a Polyglot GPU Runtime. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE Computer Society, Los Alamitos, CA, USA, 111–120. https://doi.org/10.1109/IPDPS49936.2021.00020

Daniel P Playne, Mitchell Johnson, and Kenneth A Hawick. 2009. Benchmarking GPU Devices with N-Body Simulations. *CDES* 9 (2009), 132.

Rapidsai. 2020. Issue #3203 · rapidsai/cuml. https://github.com/rapidsai/cuml/issues/3203

Rapidsai. 2021. Issue #4257 · rapidsai/cuml. https://github.com/rapidsai/cuml/issues/4257

Rapidsai. 2022. Pull Request #4069 · rapidsai/cuml. https://github.com/rapidsai/cuml/pull/4069

Rapidsai. 2023. Pull Request #5476 · rapidsai/cuml. https://github.com/rapidsai/cuml/pull/5476

Rapidsai-cuML. 2018. GitHub - rapidsai-cuml: cuML - RAPIDS Machine Learning Library. https://github.com/rapidsai/cuml

Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You Only Look Once: Unified, Real-Time Object Detection . In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, Los Alamitos, CA, USA, 779–788. https://doi.org/10.1109/CVPR.2016.91

S.-Kazem Shekofteh, Hamid Noori, Mahmoud Naghibzadeh, Holger Fröning, and Hadi Sadoghi Yazdi. 2020. cCUDA: Effective Co-Scheduling of Concurrent Kernels on GPUs. *IEEE Transactions on Parallel and Distributed Systems* 31, 4 (2020), 766–778. https://doi.org/10.1109/TPDS.2019.2944602

Markus Steinberger, Michael Kenzel, Pedro Boechat, Bernhard Kerbl, Mark Dokter, and Dieter Schmalstieg. 2014. Whippletree: Task-Based Scheduling of Dynamic Workloads on the GPU. *ACM Trans. Graph.* 33, 6, Article 228 (nov 2014), 11 pages. https://doi.org/10.1145/2661229.2661250

CUDA Streams. 2015. Streams Simplify Concurrency. https://developer.nvidia.com/blog/gpu-pro-tip-cuda-7-streams-simplify-concurrency/. Retrieved: 2023-08-08.

Yifan Sun, Xiang Gong, Amir Kavyan Ziabari, Leiming Yu, Xiangyu Li, Saoni Mukherjee, Carter McCardwell, Alejandro Villegas, and David Kaeli. 2016. Hetero-mark, a benchmark suite for CPU-GPU collaborative computing. In *2016 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, IEEE, nill, 1–10.

Tensorflow. 2023. https://github.com/tensorflow/tensorflow/commit/9d1262082e761cd85d6726bcbdfdef331d6d72c6

Han Wang, Linfeng Zhang, Jiequn Han, and Weinan E. 2018. DeePMD-kit: A deep learning package for many-body potential energy representation and molecular dynamics. *Comput. Phys. Comm.* 228 (jul 2018), 178–184. https://doi.org/10.1016/j.cpc.2018.03.016

John Whaley and Martin Rinard. 1999. Compositional pointer and escape analysis for Java programs. In *Proceedings of the 14th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Denver, Colorado, USA) *(OOPSLA '99)*. Association for Computing Machinery, New York, NY, USA, 187–206. https://doi.org/10.1145/320384.320400

Robert Paul Wilson. 1998. *Efficient, context-sensitive pointer analysis for c programs.* Ph. D. Dissertation. Stanford, CA, USA. Advisor(s) Lam, Monica S. AAI9837324.

Jingyue Wu, Artem Belevich, Eli Bendersky, Mark Heffernan, Chris Leary, Jacques Pienaar, Bjarke Roune, Rob Springer, Xuetian Weng, and Robert Hundt. 2016. Gpucc: An Open-Source GPGPU Compiler. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization* (Barcelona, Spain) *(CGO '16)*. Association for Computing Machinery, New York, NY, USA, 105–116. https://doi.org/10.1145/2854038.2854041

Sen Ye, Yulei Sui, and Jingling Xue. 2014. Region-Based Selective Flow-Sensitive Pointer Analysis. In *Static Analysis*, Markus Müller-Olm and Helmut Seidl (Eds.). Springer International Publishing, Cham, 319–336.

Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. 2018. Open3D: A Modern Library for 3D Data Processing. *arXiv:1801.09847* (2018).